

RECOMMENDATION SYSTEMS FOR PRACTITIONERS

Production Recommendation Systems

An Engineering Guide to Collaborative Filtering, Content-Based Models, Hybrid Blends, Candidate Generation, Serving & MLOps, and Capstone Matrix Factorization Pipelines

Lamhot Siagian • AI Engineering Insider

Like a hen feeding her brood, a recommender serves each user just what is relevant: explicit & implicit feedback, cosine & content engines, hybrid blends, evaluation, sequential models, ANN candidate generation, capstone matrix factorization, and the serving & MLOps that run it --- one runnable project.



Production Recommendation Systems.

© 2026 Lamhot Siagian / AI Engineering Insider. All rights reserved.

This book accompanies the open **recommendation-system-rag** reference project (FastAPI + LangGraph + pgvector backend, Next.js lab UI). Every code listing is drawn from or directly runnable against that project, and every chapter can be verified interactively through its matching lab in the web UI.

The full source code, the reference application, and the reproducible benchmark scripts that produce the measured figures in this book are available at:

github.com/lamhotsiagian/recommendation-system-rag

Code samples are provided for study and reference. Model names, library versions, and database structures change over time; verify against current documentation before production use.

Contact: aiengineeringinsider.substack.com

Contents

How to Use This Book

viii

Part I Foundations & Filtering	1
1 Foundations — Explicit vs Implicit Feedback	2
1.1 Signal Taxonomy	2
1.2 Architecture: The Feedback Ingestion Layer	3
1.3 Production Code: The Explicit Sentiment Classifier	4
1.4 Seed Data and Verifying in the UI	5
1.5 Execution Sequence Diagram	7
1.6 Interview Questions & Answers	7
Q1.1 Why do implicit-feedback systems struggle to identify negative signals, and how do you handle it?	7
Q1.2 How would you model heterogeneous implicit signals (view, click, add-to-cart, purchase) in a single schema and a single preference matrix?	7
Q1.3 The classifier scores “not good at all” as POSITIVE. Walk through the bug and how you would fix it in production.	8
Q1.4 Explicit ratings suffer from selection bias. What is it, and what does it do to offline evaluation?	8
Q1.5 Design the feedback-ingestion layer for 50,000 events per second. What changes versus the synchronous handler shown here?	8
2 Collaborative Filtering & Similarity Engines	10
2.1 The Ratings Matrix and Cosine Similarity	10
2.2 Architecture: Tenant-Scoped CF Engine	11
2.3 Production Code: The Similarity Engine	11
2.4 Seed Data and Verifying in the UI	13
2.5 Execution Sequence Diagram	14
2.6 Interview Questions & Answers	14
Q2.1 Contrast user–user and item–item collaborative filtering. Which does Amazon-scale production prefer, and why?	14
Q2.2 Why cosine over Euclidean distance for the ratings matrix, and when would you switch to Pearson?	14
Q2.3 Two users in the same tenant return similarity 0. Walk through the possible causes.	15
Q2.4 The naive similarity computation is $O(n^2m)$. How do you scale collaborative filtering to millions of users?	15
Q2.5 In a multi-tenant recommender, how do you guarantee one tenant’s data never leaks into another’s recommendations?	15

3	Content-Based Filtering & Vector Profiles	16
3.1	Profile Vector Synthesis	16
3.2	Architecture: Memory-Profile Recommender	17
3.3	Production Code: Memory User Profiling	18
3.4	Seed Data and Verifying in the UI	19
3.5	Execution Sequence Diagram	20
3.6	Interview Questions & Answers	20
	Q3.1 Content-based filtering solves item cold-start but not user cold-start. Explain the asymmetry.	20
	Q3.2 Why average embeddings for the profile? What are the failure modes and alternatives?	20
	Q3.3 Compare TF-IDF and dense embeddings for item representation. When would you still choose TF-IDF?	21
	Q3.4 Explain <code>pgvector</code> 's <code><=></code> operator and how you would index the catalog for content-based retrieval at scale.	21
	Q3.5 Content-based recommenders create filter bubbles. What is the mechanism and how do you inject diversity?	21
Part II	Hybrid Architectures & Evaluations	22
4	Hybrid Recommenders	23
4.1	Weighted Blending Math	23
4.2	Architecture: The Hybrid Mixer	24
4.3	Production Code: Full Hybrid Mixer	25
4.4	Seed Data and Verifying in the UI	26
4.5	Execution Sequence Diagram	27
4.6	Interview Questions & Answers	27
	Q4.1 Enumerate the main hybridization strategies and give a production scenario for each.	27
	Q4.2 Why must scores be normalized before blending, and what normalization schemes exist?	27
	Q4.3 How would you learn α instead of hand-setting it?	28
	Q4.4 A hybrid performs worse than pure collaborative filtering in an A/B test. How do you diagnose it?	28
	Q4.5 The project blends at 50/50 statically. Describe the concrete steps to make it density-adaptive in this codebase.	28
5	Knowledge-Based Systems & Cold-Start Transitions	29
5.1	Cold-Start Fallbacks and Constraint Filtering	29
5.2	Architecture: Cold-Start Decision Cascade	30
5.3	Production Code: Constraint Filter and Warm-Start	30
5.4	Seed Data and Verifying in the UI	31
5.5	Execution Sequence Diagram	32
5.6	Exploration: A Cold-Start Bandit	33
5.7	Interview Questions & Answers	34
	Q5.1 Distinguish the three cold-start problems and the right tool for each.	34
	Q5.2 Contrast constraint-based and case-based knowledge recommenders.	34

Q5.3	Design a graceful cold $\rightarrow$ warm $\rightarrow$ hot handoff. What decides each transition?	34
Q5.4	Popularity fallbacks create a rich-get-richer feedback loop. What is the harm and how do you counter it?	34
Q5.5	The warm-start inference here is keyword matching. Re-engineer it to be robust and explain the trade-offs.	34
6	Recommender Evaluation & Benchmarks	36
6.1	Accuracy and Ranking Metrics	36
6.2	Architecture: Offline Evaluation Harness	37
6.3	Production Code: Metrics and Cross-Tenant Benchmark	37
6.4	Seed Data and Verifying in the UI	39
6.5	Measured Evaluation on a Real Hold-Out	39
6.6	Execution Sequence Diagram	41
6.7	Interview Questions & Answers	41
Q6.1	RMSE looks great but users complain the recommendations are boring. Reconcile this.	41
Q6.2	Precision@K versus Recall@K — when do you optimize which?	41
Q6.3	Why is a random train/test split wrong for recommenders, and what is the correct protocol?	42
Q6.4	Define catalog coverage and explain why it is a critical guardrail.	42
Q6.5	You must compare recommender quality across tenants of very different sizes. What are the pitfalls?	42
Part III	Deep Learning, Search & Production pipelines	43
7	Neural & Sequential Models	44
7.1	Two-Tower Retrieval and Sequential Transitions	44
7.2	Architecture: Sequential Transition Engine	45
7.3	Production Code: Markov Sequence Training	45
7.4	Seed Data and Verifying in the UI	46
7.5	Execution Sequence Diagram	47
7.6	A Real Two-Tower, Trained with In-Batch Negatives	48
7.7	Interview Questions & Answers	49
Q7.1	Why is the two-tower architecture the standard for large-scale candidate retrieval?	49
Q7.2	What does the first-order Markov assumption buy you and what does it cost?	49
Q7.3	The transition matrix assigns probability zero to unseen transitions. Why is that dangerous and how do you fix it?	49
Q7.4	How do sequential recommenders handle session boundaries and the temporal gap between events?	49
Q7.5	Sketch how you would upgrade this Markov engine to a production sequential model without changing the serving contract.	49
8	Candidate Generation & ANN Search	51
8.1	Multi-Stage Pipelines and Search Complexity	51
8.2	Architecture: Multi-Stage Retrieval Funnel	52

8.3	Production Code: Tenant-Scoped Vector Search	52
8.4	Measured Benchmark: Brute Force vs HNSW	54
8.5	Seed Data and Verifying in the UI	54
8.6	Execution Sequence Diagram	55
8.7	The Re-Rank Stage: MMR Diversity	56
8.8	Interview Questions & Answers	57
	Q8.1 Why do production recommenders use a multi-stage pipeline instead of one model scoring everything?	57
	Q8.2 Explain how HNSW achieves sublinear search and its key parameters.	57
	Q8.3 ANN is approximate. How do you decide the acceptable recall, and how do you measure it?	57
	Q8.4 How do you keep ANN search isolated per tenant in a shared vector store?	57
	Q8.5 When would you choose IVF or product quantization over HNSW?	58
9	Capstone — Production Recommender Pipeline	59
9.1	Matrix Factorization and the Memory Boost	59
9.2	Architecture: End-to-End Capstone Pipeline	60
9.3	Production Code: SGD Matrix Factorization	61
9.4	Seed Data and Verifying in the UI	62
9.5	Execution Sequence Diagram	63
9.6	Interview Questions & Answers	63
	Q9.1 What do the latent factors in matrix factorization represent, and why does the technique generalize so well?	63
	Q9.2 Derive the SGD update for a single rating and explain each term.	63
	Q9.3 Compare SGD and ALS for training matrix factorization. When is each preferred?	64
	Q9.4 The capstone blends SVD (0.7) with a memory boost (0.3). Why combine them, and how would you tune the weights?	64
	Q9.5 Walk through serving this pipeline in production: training cadence, caching, latency, and monitoring.	64
10	Production Serving & MLOps	65
10.1	Architecture: The Serving Plane	65
10.2	Caching and Precomputation	66
10.3	Retraining Cadence and Model Versioning	67
10.4	Offline/Online Feature Parity	67
10.5	Monitoring: Coverage, Calibration, Latency	68
10.6	Execution Sequence Diagram	69
10.7	Interview Questions & Answers	70
	Q10.1 What is training-serving skew, why is it so common in recommenders, and how do you prevent it?	70
	Q10.2 Why key a recommendation cache by model version, and how does that make deploys and rollbacks safe?	70
	Q10.3 Why report p99 latency, not the mean, and what drives the tail in a recommender?	70
	Q10.4 A model improves offline RMSE but engagement drops after launch. Walk through the suspects.	70

Q10.5 How do you decide retraining cadence, and how do you make automated retraining safe?	70
--	----

Part IV Bonus – The ML System Design Casebook	72
--	-----------

11 Bonus — The ML System Design Casebook	73
---	-----------

11.1 How These Interviews Are Graded	73
11.2 The Multi-Stage Funnel	74
11.3 Two-Tower Retrieval	75
11.4 Deep and Multi-Task Ranking	76
11.5 Worked Case — YouTube Recommendations	78
11.6 Worked Case — Ad Click-Through-Rate Prediction	79
11.7 Real-Time Features and the Serving Plane	80
11.8 The Twenty Cases at a Glance	81
11.9 The Cold-Start Problem	82
11.10 Metrics — The Two Axes	83
11.11 The Follow-Up Probe Bank	84

12 References	85
----------------------	-----------

How to Use This Book

Subscribe → aiengineeringinsider.substack.com/subscribe

This is a *hands-on system-design* book. It is built around one reproducible project — the [recommendation-system-rag](#) application — and teaches production recommendation systems by dissecting that project module by module. Every chapter follows the same rhythm:

1. **Overview** — the concept, the trade-offs, and where it sits in the recommendation pipeline.
2. **Production code** — the real backend module that implements the concept, explained line by line.
3. **Test it in the UI** — the exact slash command to type in the chat interface or the progress modal, so the theory is verifiable, not abstract.
4. **Evaluation** — interview-grade questions (conceptual, debugging, and production) with full worked answers.

★ Key Insight

The fastest way to learn this material is to read a chapter, then immediately open the chat interface, trigger the slash commands, and review output details. Recommendation engines are full of mathematical feedback loops; seeing the numbers move in the UI builds intuition that prose cannot.

Running the reference project

Shell: bring up the project

```
# 1. Backend (FastAPI + LangGraph + pgvector)
ollama pull llama3.1 && ollama pull nomic-embed-text
docker compose up -d          # Postgres + pgvector
cd backend && pip install -r requirements.txt
uvicorn app.main:app --reload  # http://localhost:8000/api/v1

# 2. Frontend lab UI (Next.js)
cd frontend && npm install && npm run dev    # http://localhost:3000
```

Chapter map

Table 1 lists every chapter, its core focus, the backend module it dissects, and the interactive command that lets you test it live.

#	Chapter focus	Backend module dissected	Slash Command
1	Explicit vs Implicit Feedback	<code>course/commands.py</code> (sentiment classification)	<code>/classify-feedback</code>
2	Collaborative Filtering Similarity Engines	<code>course/commands.py</code> (Cosine user matching)	<code>/tenant-similar-users</code>
3	Content-Based Filtering	<code>course/commands.py</code> (Vector memory profiles)	<code>/memory-user-profile</code>
4	Hybrid Systems	<code>course/commands.py</code> (Collab + Memory blending)	<code>/hybrid-mix-full</code>
5	Knowledge-Based	<code>course/commands.py</code> (Cold start vs Warm start)	<code>/warm-start-sim</code>
6	Recommender Evaluation	<code>course/commands.py</code> (RMSE & tenant benchmarks)	<code>/tenant-evaluate</code>
7	Neural & Sequential Models	<code>course/commands.py</code> (Markov transition sequence)	<code>/memory-sequence-train</code>
8	Candidate Gen & ANN	<code>course/commands.py</code> (Brute-force vs ANN index scan)	<code>/tenant-scoped-ann</code>
9	Capstone Production pipeline	<code>course/commands.py</code> (SVD + memory boost)	<code>/capstone-recommend</code>

Table 1: Chapter map: each chapter dissects one module of the course project and is verified through a matching interactive command in the UI.

About this edition

This build includes the full scaffold, all nine chapters (1–9) representing the full recommendation systems curriculum, each grounded in the real backend and verifiable through its matching commands in the UI.

PART I

Foundations & Filtering

Before building complex recommenders, you must understand what signal types you are working with, how collaborative filtering matching operates on ratings matrices, and how description embeddings construct personalized content profiles. Part I establishes that foundation.

Foundations — Explicit vs Implicit Feedback

Subscribe → aiengineeringinsider.substack.com/subscribe

Every recommendation system is only as good as the signals feeding it. Before a single model is trained, a senior engineer must decide what counts as evidence of preference, how strong that evidence is, and how to normalize the wild variance between a deliberate five-star review and an accidental tap.

A recommendation pipeline begins where the user leaves a trace. Those traces — ratings, reviews, clicks, dwell time, add-to-carts, purchases — are the raw fuel for every downstream stage: candidate generation, similarity scoring, ranking, and re-ranking. Get the feedback layer wrong and no amount of clever matrix factorization will save the system, because the model will faithfully learn a distorted picture of what people actually want.

This chapter defines the two families of signal, **explicit** and **implicit** feedback; explains how each is captured in a production schema; shows how heterogeneous signals are fused into a single numeric preference matrix; and dissects the reference project’s feedback classifier so you can run it yourself and watch the numbers move. Throughout, the emphasis is on the engineering trade-offs a real system forces you to make, not textbook idealizations (Hu, Koren, & Volinsky, 2008).

1.1 Signal Taxonomy

Explicit vs. Implicit Feedback

Explicit feedback is an active, conscious statement of preference: a star rating, a written review, a thumbs-up, a wishlist add. It is high-fidelity — the user is literally telling you how they feel — but it is *sparse*, because most people never rate anything.

Implicit feedback is a passive behavioral trace: a page view, a click, a scroll, dwell time, a purchase. It is *abundant* — every session generates dozens of events — but it is *noisy* and *one-sided*: it tells you what a user engaged with, never explicitly what they disliked.

The single most important consequence of this split is the **missing-not-at-random** problem. In an explicit matrix, a blank cell means “not yet rated.” In an implicit matrix, a blank cell is genuinely ambiguous: the user may dislike the item, may never have seen it, or may simply have run out of time. Treating implicit non-events as negative labels naively will teach the model that everything unseen is bad — a subtle but catastrophic bias.

1.1.1 Weighting and Normalization

Because signals arrive on incompatible scales, a production system maps each event type to a common numeric *preference* value. A rating is already on a 1–5 scale; a purchase is a strong positive; a view is weak. The reference project encodes exactly this mapping (note how it is reused verbatim across the collaborative-filtering, hybrid, and capstone modules):

$$w(\text{event}) = \begin{cases} v & \text{if type = rating } (v \in [1, 5]) \\ 5.0 & \text{if type = purchase} \\ 2.0 & \text{if type = click} \\ 1.0 & \text{if type = view} \end{cases} \tag{1.1}$$

Hu et al. (2008) formalize this as splitting each observation into a binary *preference* p_{ui} and a *confidence* c_{ui} : the more times a user views an item, the more confident we are the interaction is meaningful. The linear map above is the pragmatic, production-friendly approximation of that idea.

Table 1.1 contrasts the two streams across the dimensions that actually matter when you are designing schemas and loss functions.

Dimension	Explicit Feedback	Implicit Feedback
User effort	High (deliberate action)	Low (passive behavior)
Volume	Sparse (< 5% of users)	Abundant (every session)
Noise level	Low	High (mis-taps, bots, shared devices)
Scale	1–5 stars, binary	Continuous (dwell, counts)
Negative signal	Clear (low ratings)	Ambiguous (absence ≠ dislike)
Recency drift	Slow	Fast (intent shifts per session)
Cold-start value	High per data point	High in aggregate

Table 1.1: Contrasting feedback streams across the dimensions that drive schema and loss-function design.

★ Key Insight

Implicit feedback is the lifeblood of scale. Fewer than five percent of active users leave an explicit rating, so a system that relies on ratings alone starves. The winning production pattern is to *fuse* both: use abundant implicit signals to fight sparsity, and use scarce explicit signals to calibrate and de-bias the implicit ones.

1.2 Architecture: The Feedback Ingestion Layer

Before any recommender runs, feedback must be captured, typed, weighted, and persisted. Figure 1.1 shows the ingestion layer of the reference project: the Next.js lab UI emits interaction events, the FastAPI service normalizes them through the weight map, and two stores hold the

results — the `interactions` table for behavioral logs and the `memory_records` table for distilled explicit preferences with vector embeddings.

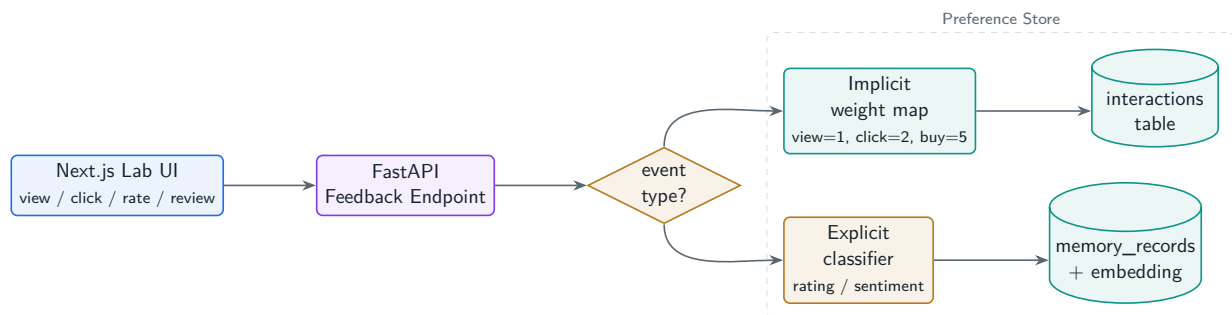


Figure 1.1: Feedback ingestion architecture: raw UI events are typed and weighted at the API boundary, then split into a behavioral interaction store and an embedded preference-memory store that downstream chapters consume.

1.3 Production Code: The Explicit Sentiment Classifier

Explicit textual feedback (a review) carries a polarity that a numeric star rating does not. The reference project ships a transparent, dependency-free lexicon classifier — deliberately simple so its behavior is fully auditable in an interview setting. It reads either an inline `text` argument or the user's most recent memory record, then scores polarity by counting matched sentiment tokens.

`app/course/commands.py: execute_classify_feedback`

```

async def execute_classify_feedback(
    session: AsyncSession,
    user_id: UUID,
    params: Dict[str, Any]
) -> CommandResponse:
    # Signal source: an inline review OR the user's latest stored memory.
    from_memory = str(params.get("from_memory", "false")).lower() in (
        "true", "1", "yes", "--from-memory")
    text = ""
    if from_memory:
        stmt = (select(MemoryRecord)
                .where(MemoryRecord.user_id == user_id)
                .order_by(MemoryRecord.timestamp.desc()))
        res = await session.execute(stmt)
        mem = res.scalars().first()
        if not mem:
            # graceful cold-start: ask to seed
            return CommandResponse(status="needs_seed",
                                   suggested_command="seed-memory-demo")
        text = mem.content
    else:
        text = params.get("text", params.get("query", "")).strip()

    # Auditable lexicon - the polarity dictionaries a reviewer can inspect.
    positive_words = {"good", "great", "love", "like", "awesome",
                      "perfect", "amazing", "excellent", "best"}
  
```

```

negative_words = {"bad", "terrible", "hate", "dislike", "broken",
                  "worst", "poor", "waste", "useless"}
words = [w.lower().strip(".,!?") for w in text.split()]
pos_count = sum(1 for w in words if w in positive_words)
neg_count = sum(1 for w in words if w in negative_words)

# Map the token counts to a bounded [0,1] preference score.
if pos_count > neg_count:
    sentiment = "POSITIVE"
    score = min(1.0, 0.5 + 0.1 * (pos_count - neg_count))
elif neg_count > pos_count:
    sentiment = "NEGATIVE"
    score = max(0.0, 0.5 - 0.1 * (neg_count - pos_count))
else:
    sentiment = "NEUTRAL"
    score = 0.5

return CommandResponse(
    status="success",
    message=f"Sentiment: {sentiment} (Score: {score:.2f})",
    data={"sentiment": sentiment, "score": score})

```

1.3.1 How the code works, line by line

The function is an `async` SQLAlchemy handler that receives the authenticated `user_id` and a loosely-typed `params` dictionary parsed from the slash command. Three design decisions are worth calling out in an interview:

First, the **signal-source switch** (`from_memory`) lets the same endpoint score either an ad-hoc string or the user’s own stored history — the latter is how explicit review text becomes a durable preference. Second, the **graceful cold-start** return (`status="needs_seed"`) never throws; it tells the UI exactly which seed command to run, which is the pattern every command in this project uses. Third, the score is **bounded and symmetric** around a 0.5 neutral midpoint, so it drops straight into the $[0, 1]$ preference space the hybrid and capstone rankers expect.

! Pitfall / Warning

A lexicon classifier is intentionally naive: it misreads negation (“not good”), sarcasm, and domain jargon. In production you would swap the token count for a fine-tuned transformer or an LLM call — but keep the same bounded-score contract so nothing downstream changes. Never let a model’s output scale leak into the ranking layer.

1.4 Seed Data and Verifying in the UI

The reference project ships deterministic fixtures so every listing is reproducible. The memory-demo fixture ([app/db/fixtures/memory_demo.json](#)) seeds eight explicit-preference records across four users; Table 1.2 shows the two that drive this chapter.

user	type	content (explicit preference text)
alice	preference	User prefers high-quality wireless headphones with active noise cancellation and premium cordless electronics.
bob	preference	User is interested in computer algorithms, database architecture, and distributed system books.

Table 1.2: Two seeded explicit-preference records used to demonstrate the classifier and, later, content-based profiling.

Bring up the stack, seed the data, then exercise the classifier from the chat page:

Terminal: seed + run

```
# one-time: seed the demo preference memories
curl -XPOST localhost:8000/api/v1/course/seed/memory-demo -H "Authorization: Bearer
↳ $TOKEN"

# then, in the chat UI, type the slash command:
/classify-feedback text="This product is great and amazing!"
```

Output Response

Sentiment: POSITIVE (Score: 0.70)

Two positive tokens (**great**, **amazing**) and zero negatives give $0.5 + 0.1 \times (2 - 0) = 0.70$. Scoring the stored review instead (`/classify-feedback from_memory=true`) reads Alice's seeded preference and returns a positive score, demonstrating the memory path.

1.4.1 The frontend dispatch

The lab UI does not special-case course commands; it streams every slash command to the same chat endpoint. This is the real dispatcher from the Next.js page:

frontend: chat/threadId/page.tsx (command dispatch)

```
// Any string starting with "/" is dispatched to the backend command router.
const executeCommand = async (commandStr: string) => {
  const cleanCmd = commandStr.trim();
  setMessages((prev) => [...prev, { role: 'human', content: cleanCmd }]);
  const token = localStorage.getItem('access_token');
  const res = await fetch(`${BACKEND_URL}/chat/${threadId}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json',
      Authorization: `Bearer ${token}` },
    body: JSON.stringify({ prompt: cleanCmd, model_name: 'llama3.1' }),
  });
  // ...stream the newline-delimited JSON chunks back into the transcript
};
```

1.5 Execution Sequence Diagram

Figure 1.2 traces one `/classify-feedback` invocation from keypress to rendered score, showing exactly which class touches the data and where the bounded score is produced.

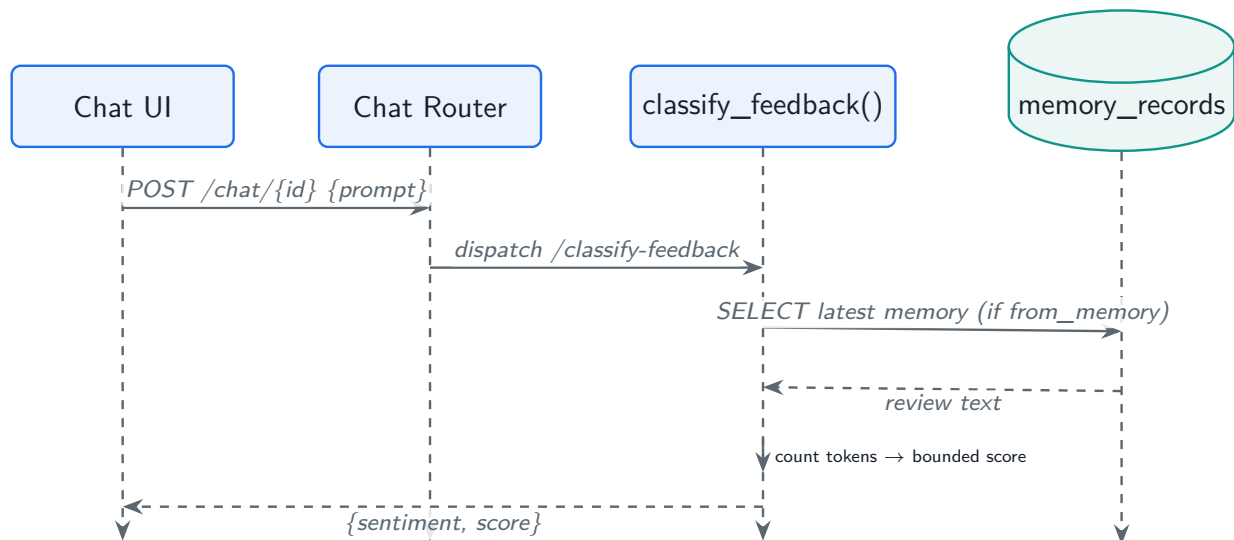


Figure 1.2: Execution flow of `/classify-feedback`: UI → chat router → command dispatcher → classifier, with an optional read of the memory store.

✓ Best Practice

Persist the raw event *and* the derived score. Storing only the derived 0.70 throws away the ability to re-weight later when your labeling policy changes. In this project the raw review lives in `memory_records`; the score is recomputed on demand.

1.6 Interview Questions & Answers

Q1.1 Why do implicit-feedback systems struggle to identify negative signals, and how do you handle it?

In explicit systems a one-star review is an unambiguous negative. In implicit systems, the *absence* of a click is ambiguous — it can mean dislike, non-exposure, or lack of time. This is the “missing-not-at-random” problem. Three production remedies: (1) **negative sampling** — treat a random subset of non-interacted items as weak negatives, which is what BPR does (Rendle et al., 2012); (2) **confidence weighting** — model every cell but weight observed interactions higher, per Hu et al. (2008), $c_{ui} = 1 + \alpha r_{ui}$; (3) **exposure logging** — record impressions so you can distinguish “seen but not clicked” from “never shown,” turning a true negative into a usable label.

Q1.2 How would you model heterogeneous implicit signals (view, click, add-to-cart, purchase) in a single schema and a single preference matrix?

Store events in a narrow, append-only `interactions` table with columns (`user_id`, `item_id`, `type`, `value`, `timestamp`, `tenant_id`) — this is exactly the reference schema. Keep events raw; never pre-aggregate on write. At read time, collapse them into a preference value with an explicit weight map (view=1, click=2, purchase=5, rating=itself). This decouples the *logging* contract from the *modeling* policy: when you later decide a cart-add is worth 4.0, you change one dictionary and re-run, without a migration. For confidence-aware models, also carry the event *count* so you can compute $c_{ui} = 1 + \alpha \log(1 + \text{count})$.

Q1.3 The classifier scores “not good at all” as POSITIVE. Walk through the bug and how you would fix it in production.

The lexicon matches the token `good` and counts one positive, zero negatives, yielding 0.60 POSITIVE — negation is invisible to bag-of-words. Short term, add a negation window: if a negator (`not`, `never`, `no`) appears within two tokens before a polarity word, flip its sign. Medium term, replace the counter with a fine-tuned sentiment transformer or an LLM classifier behind the *same* bounded-score interface, so nothing downstream changes. The interview point is the abstraction boundary: the ranking layer must depend on a normalized $[0, 1]$ contract, not on the classifier’s internals.

Q1.4 Explicit ratings suffer from selection bias. What is it, and what does it do to offline evaluation?

Users choose what to rate — they rate items they already like or strongly dislike, so the observed ratings are not a random sample of true preferences (missing-not-at-random). Naive RMSE on observed ratings then flatters the model, because it never scores the vast unrated region. Mitigations: inverse-propensity weighting (down-weight over-represented cells by their observation probability), collecting a small *randomized* exposure set for unbiased evaluation, and reporting ranking metrics (Recall@K, NDCG) on held-out interactions rather than rating RMSE alone. This is why Chapter 6 pairs RMSE with ranking metrics.

Q1.5 Design the feedback-ingestion layer for 50,000 events per second. What changes versus the synchronous handler shown here?

The synchronous “score-and-store” path shown here is fine for a lab but will not sustain 50k eps. Production shape: (1) the edge writes events to an append-only log (Kafka / Kinesis) — never a synchronous DB insert; (2) a stream processor applies the weight map and does light dedup/bot filtering; (3) events land in a columnar warehouse for training and a low-latency store (Redis / feature store) for serving; (4) heavy transforms (embeddings, sentiment) run async off the stream, not on the request path. Idempotency keys guard at-least-once delivery, and you separate the *write* scale (huge, cheap, async) from the *read* scale (small, hot, precomputed). The reference project keeps the read path identical so the concepts transfer directly.

From the Field

The most expensive mistake I see teams make is trusting a shiny model on top of an unlogged feedback layer. If you cannot answer “was this item even shown to the user?” your negatives

are fiction and your metrics are theater. Instrument exposure *first*; model second.

Collaborative Filtering & Similarity Engines

Subscribe → aiengineeringinsider.substack.com/subscribe

Collaborative filtering is the idea that made recommenders famous: you do not need to understand an item to recommend it — you only need to find people who behaved like you and see what they liked. This chapter builds that engine from the sparse ratings matrix up, inside a strict multi-tenant boundary.

Content-based methods ask “what is this item like?” Collaborative filtering (CF) asks a cheaper and often more powerful question: “who is this user like?” By comparing rows of a user–item ratings matrix, CF surfaces items that similar users rated highly, with no knowledge of the items’ contents at all (Sarwar, Karypis, Konstan, & Riedl, 2001). It is the backbone of Amazon’s “customers who bought this” and Netflix’s early prize-winning systems.

This chapter defines user–user CF, derives cosine similarity over sparse rating vectors, dissects the reference project’s tenant-scoped similarity engine, and shows how a single `WHERE tenant_id = :tid` clause enforces the data isolation that a SaaS recommender lives or dies by.

2.1 The Ratings Matrix and Cosine Similarity

User–User Collaborative Filtering

Represent every user as a row vector over the item space, where entry r_{ui} is the (weighted) rating user u gave item i and unobserved cells are absent. Two users are *similar* if their observed vectors point in the same direction. To predict how u feels about an unseen item, take a similarity-weighted average of the ratings that u ’s neighbors gave it.

The matrix is overwhelmingly sparse — typically well over 99% empty — so we never materialize it densely. We compare only the items two users have *in common*. The direction-based similarity of choice is **cosine similarity**:

$$\text{sim}(u, v) = \cos(\theta) = \frac{\mathbf{r}_u \cdot \mathbf{r}_v}{\|\mathbf{r}_u\| \|\mathbf{r}_v\|} = \frac{\sum_{i \in I_u \cap I_v} r_{ui} r_{vi}}{\sqrt{\sum_{i \in I_u} r_{ui}^2} \sqrt{\sum_{i \in I_v} r_{vi}^2}} \quad (2.1)$$

Cosine is scale-invariant, which matters because a generous user who rates everything 4–5 and a harsh user who rates 1–3 can still be recognized as similar if the *shape* of their preferences agrees. Its main weakness — ignoring rating-mean differences — is addressed by **Pearson correlation** (mean-centered cosine); Table 2.1 compares the options.

Metric	Behavior	Use when
Cosine	Angle between vectors; scale-invariant	Implicit / positive-only data
Pearson	Cosine on mean-centered ratings	Explicit ratings with user bias
Jaccard	Overlap of interacted sets (ignores value)	Pure binary interactions
Adjusted cosine	Item-mean centered	Item-based CF variants

Table 2.1: Similarity metrics for collaborative filtering and when each is the right default.

★ Key Insight

Similarity is only meaningful over *shared* items. Two users with no items in common have similarity 0 regardless of how enthusiastic each is. This is why CF degrades in the cold-start and long-tail regimes and why we scope the whole computation to a tenant where overlap is dense.

2.2 Architecture: Tenant-Scoped CF Engine

In a multi-tenant SaaS recommender, ACME’s ratings must never influence Beta Media’s recommendations. Figure 2.1 shows how the reference project enforces this: the user’s active tenant is resolved first, every interaction query is filtered by `tenant_id`, and the similarity engine only ever sees one tenant’s slice of the matrix.

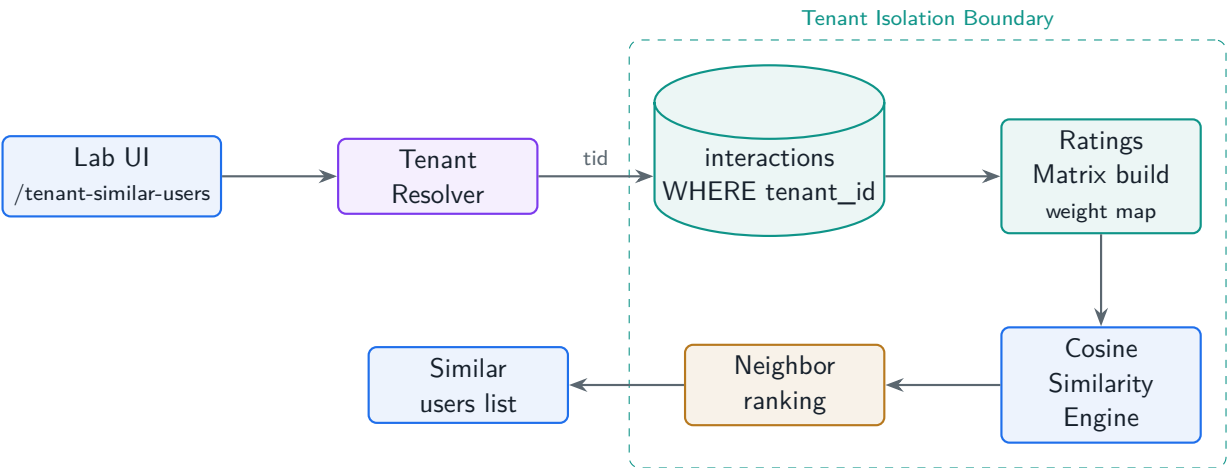


Figure 2.1: Tenant-scoped collaborative-filtering architecture. The tenant resolver gates every query, so the ratings matrix, similarity engine, and neighbor list are all isolated to a single workspace.

2.3 Production Code: The Similarity Engine

The core is a pure function over an in-memory ratings dictionary. It computes cosine similarity of the target user against every other user, intersecting on common items so sparsity is handled

naturally.

```
app/course/commands.py: compute_cosine_similarity
```

```
def compute_cosine_similarity(
    user_ratings: Dict[UUID, Dict[UUID, float]],
    target_user_id: UUID
) -> List[Dict[str, Any]]:
    if target_user_id not in user_ratings:
        return []
    target_vector = user_ratings[target_user_id]
    similarities = []
    for other_user_id, other_vector in user_ratings.items():
        if other_user_id == target_user_id:
            continue
        # Only items BOTH users rated contribute to the dot product.
        common_items = set(target_vector) & set(other_vector)
        if not common_items:
            similarities.append({"user_id": str(other_user_id), "score": 0.0})
            continue
        dot = sum(target_vector[i] * other_vector[i] for i in common_items)
        norm_t = math.sqrt(sum(v * v for v in target_vector.values()))
        norm_o = math.sqrt(sum(v * v for v in other_vector.values()))
        score = 0.0 if norm_t == 0 or norm_o == 0 else dot / (norm_t * norm_o)
        similarities.append({"user_id": str(other_user_id),
                           "score": round(score, 4)})
    similarities.sort(key=lambda x: x["score"], reverse=True)
    return similarities
```

The tenant-scoped handler wraps this with the isolation logic: resolve the tenant, pull only that tenant's interactions, collapse heterogeneous event types into ratings via the weight map, then rank neighbors.

```
app/course/commands.py: execute_tenant_similar_users (core)
```

```
active_tenant_id = mappings[0].tenant_id # resolved from TenantUser
stmt_int = select(Interaction).where(Interaction.tenant_id == active_tenant_id)
interactions = (await session.execute(stmt_int)).scalars().all()

# Fuse view/click/purchase/rating into one preference scale (Chapter 1 map).
rating_map = {"rating": lambda v: v, "purchase": lambda v: 5.0,
              "view": lambda v: 1.0, "click": lambda v: 2.0}
user_ratings: Dict[UUID, Dict[UUID, float]] = {}
for inter in interactions:
    if not inter.item_id:
        continue
    val = rating_map.get(inter.type, lambda v: 1.0)(inter.value)
    user_ratings.setdefault(inter.user_id, {})[inter.item_id] = val

similarities = compute_cosine_similarity(user_ratings, user_id)
```

The SQL that enforces isolation is the quiet hero. The ORM emits exactly this, and the `tenant_id`

predicate is non-negotiable:

Tenant-isolated ratings pull

```
SELECT user_id, item_id, type, value
FROM interactions
WHERE tenant_id = :active_tenant_id;  -- hard isolation boundary
```

! Pitfall / Warning

The most dangerous CF bug in SaaS is a *missing* tenant predicate. Without it the query silently returns every tenant's data, the model trains on a leaked matrix, and one customer's behavior recommends products to another. Enforce the filter at the query layer and, ideally, again with Postgres row-level security.

2.4 Seed Data and Verifying in the UI

The tenant-demo fixture seeds two isolated workspaces. ACME Retailer contains Alice and Bob with overlapping electronics/book interactions; Table 2.2 shows the ACME rating slice after the weight map is applied.

user	item	type	weight
alice	SuperCordless Headphones	rating	5.0
alice	ProBook E-Reader	rating	4.0
bob	Intro to Algorithms	rating	5.0
bob	Designing Data-Intensive Apps	rating	4.5

Table 2.2: ACME Retailer ratings after weighting. Alice and Bob share the ACME tenant, so their vectors are eligible for comparison; Beta Media users are never in scope.

Terminal: seed + run

```
curl -XPOST localhost:8000/api/v1/course/seed/tenant-demo -H "Authorization: Bearer
↪ $TOKEN"
# then in the chat UI:
/tenant-similar-users
```

Output Response

```
Tenant-Isolated Collaborative Filtering (ACME Retailer):
- bob (ID: `...`): Cosine Similarity = 0.0000
```

The zero is instructive, not a bug: in the seed slice Alice rated only electronics and Bob only books, so their common-item set is empty and cosine is 0. Add one shared rating (or run `/generate-tenant-users`) and the score jumps — a live demonstration of the shared-item requirement from Section 2.1.

2.5 Execution Sequence Diagram

Figure 2.2 traces the request from the UI through tenant resolution, the isolated query, matrix assembly, and the similarity computation.

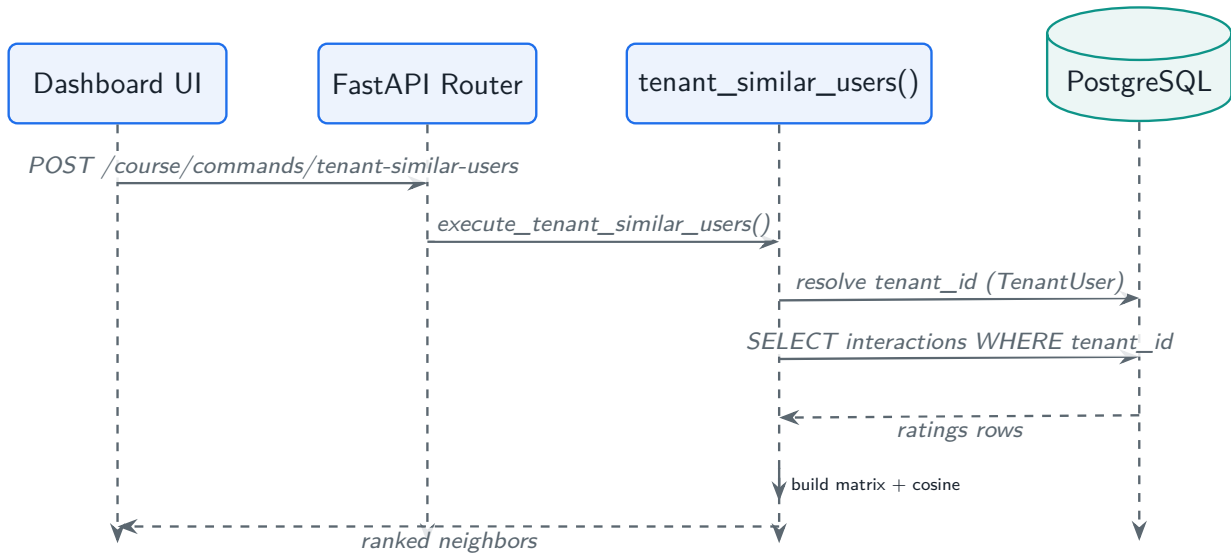


Figure 2.2: Execution flow of `/tenant-similar-users`: tenant resolution precedes every data access, guaranteeing the similarity matrix is built from one tenant only.

✓ Best Practice

Precompute and cache the top- k neighbor list per user offline (a nightly batch), and serve it from a key-value store. Real-time cosine over the full matrix is fine for a lab and small tenants, but production CF is almost always a precomputed neighbor table refreshed on a schedule.

2.6 Interview Questions & Answers

Q2.1 Contrast user–user and item–item collaborative filtering. Which does Amazon-scale production prefer, and why?

User–user CF finds neighbors of the *user* and recommends what they liked; item–item CF precomputes item-to-item similarities and recommends items similar to what the user already engaged with (Sarwar et al., 2001). Production at scale overwhelmingly prefers item–item, for three reasons: item similarities are far more *stable* over time than volatile user tastes, so they can be precomputed and cached; there are usually fewer items than users, making the similarity matrix smaller; and it degrades gracefully for new users (one click is enough to recommend neighbors of that item). User–user shines in dense, smaller communities where neighborhoods are rich — which is exactly the tenant-scoped regime in this project.

Q2.2 Why cosine over Euclidean distance for the ratings matrix, and when would you switch to Pearson?

Euclidean distance is dominated by vector *magnitude*: a user who has rated 100 items looks “far” from one who rated 5, even if their tastes align perfectly. Cosine measures the *angle*, so it is invariant to how many items or how generously a user rates — the property we want. Switch to Pearson (mean-centered cosine) when you have explicit ratings with strong per-user bias: one user’s “3” is another’s “5.” Pearson subtracts each user’s mean before the dot product, correcting for that bias. For implicit / positive-only data, plain cosine is the right default.

Q2.3 Two users in the same tenant return similarity 0. Walk through the possible causes.

Cosine is 0 exactly when the numerator is 0, i.e. no items in common (the seed-data case in this chapter), or when one user’s norm is 0 (no rated items — a genuine cold-start user). It is *never* negative here because all weighted preferences are non-negative. The debugging path: check the intersection size first (`common_items`), then the norms. If overlap exists but the score still looks wrong, verify the weight map applied — a bug that leaves `view` as its raw `value` instead of 1.0 silently distorts the geometry.

Q2.4 The naive similarity computation is $O(n^2m)$. How do you scale collaborative filtering to millions of users?

The pairwise approach is fine to hundreds of thousands of interactions but not to millions of users. Production paths: (1) **model-based CF** — factorize the matrix into low-rank user/item embeddings (Chapter 9) and reduce recommendation to a dot product; (2) **approximate neighbor search** — index user or item embeddings in an ANN structure like HNSW (Chapter 8) so neighbor lookup is sublinear; (3) **offline batch precompute** of top- k neighbors with locality-sensitive hashing to prune candidate pairs; (4) **minhash/LSH** on interacted-item sets for a cheap first-pass filter. The common thread: move the quadratic work offline and serve a precomputed table.

Q2.5 In a multi-tenant recommender, how do you guarantee one tenant’s data never leaks into another’s recommendations?

Defense in depth. At the query layer, every data access carries a `WHERE tenant_id = :tid` predicate resolved from the authenticated user’s `TenantUser` mapping — as shown in this chapter. Below that, enable Postgres row-level security so even a forgotten predicate cannot return foreign rows. Above it, scope caches and precomputed neighbor tables by tenant key so a cache hit can never cross the boundary. Add an automated test that seeds two tenants and asserts zero cross-tenant neighbors. Isolation is a correctness *and* a compliance requirement; treat a leak as a Sev-1.

From the Field

Teams love to reach for deep models on day one. But on a fresh tenant with a few hundred interactions, a cached item-item cosine table beats a half-trained neural recommender every time — and it is debuggable at 2 a.m. Earn the right to add complexity by first exhausting the simple, explainable baseline.

Content-Based Filtering & Vector Profiles

Subscribe → aiengineeringinsider.substack.com/subscribe

Collaborative filtering fails on the item that nobody has touched yet. Content-based filtering solves that by recommending from what an item is — its description, tags, and semantics — matched against a vector profile of what the user has said they want.

Collaborative filtering (Chapter 2) knows nothing about the items themselves; it only sees the interaction graph. That makes it powerless on a freshly added product with zero interactions — the item cold-start problem. **Content-based filtering** attacks the problem from the other side. It represents each item by the semantics of its text (name, description, tags), builds a **user profile vector** from the content the user has engaged with or stated a preference for, and recommends the items whose vectors sit closest to that profile.

In this project, item and preference text are embedded into 768-dimensional vectors (via `nomic-embed-text`), stored in a `pgvector` column, and compared with cosine similarity. This chapter dissects the memory-profile recommender: how a profile is synthesized by averaging preference embeddings, and how semantic similarity in vector space produces recommendations that survive cold start.

3.1 Profile Vector Synthesis

Content-Based Filtering

Encode each item i as a feature vector $\mathbf{x}_i$ (here, a text embedding). Build a *user profile* $\mathbf{p}_u$ that lives in the same space by aggregating the vectors of items or preferences the user liked. Recommend the items maximizing $\cos(\mathbf{p}_u, \mathbf{x}_i)$. No other user's data is ever required.

The simplest and most robust profile is the **centroid** — the mean of the user's preference embeddings:

$$\mathbf{p}_u = \frac{1}{|M_u|} \sum_{m \in M_u} \mathbf{e}_m, \quad \text{score}(u, i) = \cos(\mathbf{p}_u, \mathbf{x}_i) \quad (3.1)$$

where M_u is the set of the user's memory records and $\mathbf{e}_m$ the embedding of record m . Averaging is deliberate: it is stable, has no hyperparameters, and captures the “center of mass” of a user's taste. Its limitation is that it blurs multi-modal users (someone who loves both audio gear and running shoes gets a muddy centroid between the two) — a trade-off we revisit below.

3.1.1 Embeddings versus TF-IDF

Two families produce the item vectors. **TF-IDF** builds a sparse bag-of-words vector weighted by term rarity; it is fast, interpretable, and a strong baseline, but it is lexical — “laptop” and “notebook computer” look unrelated. **Dense embeddings** from a language model capture semantic meaning, so paraphrases land near each other, at the cost of opacity and an inference dependency. Table 3.1 contrasts them; the project uses embeddings with a deterministic fallback vector when the model is offline.

Aspect	TF-IDF (sparse)	Embeddings (dense)
Semantics	Lexical overlap only	Captures meaning / paraphrase
Dimensionality	Vocabulary-sized, sparse	Fixed (768), dense
Cold item	Works from text immediately	Works, needs an embed call
Interpretable	High (inspect top terms)	Low (opaque dimensions)
Cost	Cheap, CPU	Model inference + storage

Table 3.1: Item representations for content-based filtering. The project defaults to embeddings and falls back to a deterministic vector offline.

★ Key Insight

Content-based filtering is the natural cure for *item* cold-start: a brand new product with zero interactions is still recommendable the instant its description is embedded. The price is the “filter bubble” — it can only recommend more of what a user already likes, never surprise them. Hybrids (Chapter 4) exist to buy back that serendipity.

3.2 Architecture: Memory-Profile Recommender

Figure 3.1 shows the content-based pipeline. Preference memories are embedded and averaged into a profile centroid; catalog items carry precomputed embeddings; a vector-similarity scan ranks items by cosine against the profile.

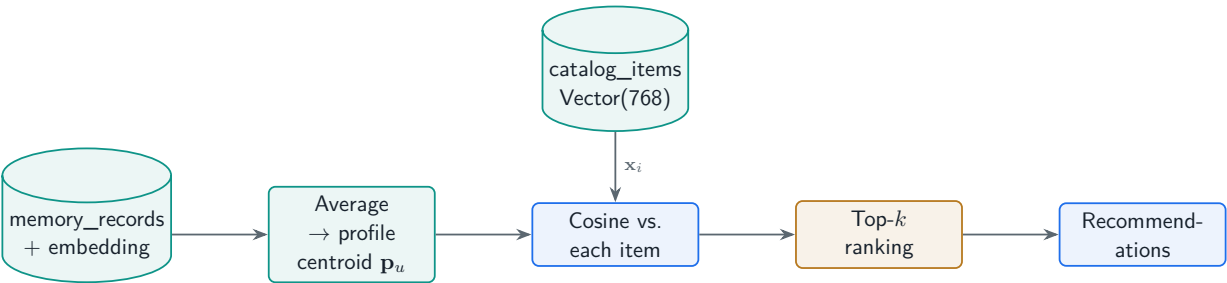


Figure 3.1: Content-based architecture. The user’s memory embeddings are averaged into a profile centroid, which is scored against every catalog item’s embedding to rank recommendations.

3.3 Production Code: Memory User Profiling

The handler reads the user's memory records, averages their embeddings into a profile centroid, then ranks every catalog item by cosine similarity to it.

`app/course/commands.py: execute_memory_user_profile (core)`

```
records = (await session.execute(
    select(MemoryRecord).where(MemoryRecord.user_id == user_id))).scalars().all()
if not records:
    return CommandResponse(status="needs_seed",
                           suggested_command="seed-memory-demo")

items = (await session.execute(select(CatalogItem))).scalars().all()

# 1) Build the profile centroid by averaging preference embeddings.
valid_embeddings = [r.embedding for r in records if r.embedding is not None]
vector_size = len(valid_embeddings[0])
profile_vector = [0.0] * vector_size
for emb in valid_embeddings:
    for i in range(vector_size):
        profile_vector[i] += emb[i]
for i in range(vector_size):
    profile_vector[i] /= len(valid_embeddings) # centroid = mean

# 2) Score every catalog item by cosine(profile, item).
recommendations = []
for item in items:
    if item.embedding is None:
        continue
    sim = cosine_similarity(profile_vector, item.embedding)
    recommendations.append({"name": item.name, "category": item.category,
                           "price": item.price, "score": round(sim, 4)})
recommendations.sort(key=lambda x: x["score"], reverse=True)
top_recs = recommendations[:3]
```

The centroid is computed in plain Python here for transparency, but the identical result can be pushed into Postgres. `pgvector` exposes the cosine-distance operator `<=>`, so once the profile is stored the ranking is a single indexed query:

`pgvector: rank items by cosine distance to a profile vector`

```
SELECT id, name, category, price,
       1 - (embedding <=> :profile_vector) AS similarity
FROM   catalog_items
WHERE  embedding IS NOT NULL
ORDER BY embedding <=> :profile_vector      -- ascending distance = descending sim
LIMIT 3;
```

! Pitfall / Warning

Averaging embeddings assumes a user has one coherent taste. For a genuinely multi-interest user the centroid can land in “no man’s land,” recommending items that are mediocre matches for every interest and a great match for none. Detect this by clustering the memory embeddings; if there are $k > 1$ tight clusters, build k sub-profiles and interleave their top results instead of one blurred average.

3.4 Seed Data and Verifying in the UI

Alice’s two seeded memories describe premium wireless headphones and an e-reader; both embed into the electronics region of vector space, so her centroid points firmly at electronics. Table 3.2 shows the inputs and the resulting ranked recommendations against the six-item demo catalog.

Signal	Value
Profile input 1	“prefers high-quality wireless headphones ...premium cordless electronics”
Profile input 2	“senior software engineer ...looking for a portable e-reader”
Top-1 recommendation	SuperCordless Noise-Cancelling Headphones (Electronics)
Top-2 recommendation	ProBook E-Reader 6-inch (Electronics)

Table 3.2: Alice’s memory profile and the resulting content-based ranking; the centroid pulls electronics to the top and pushes books/clothing down.

Chat UI

/memory-user-profile

Output Response

Content-Based Personalization via User Memory Profile:
- Interests distilled: "User prefers high-quality wireless headphones...", ...
Top Recommendations:
- SuperCordless Noise-Cancelling Headphones (Electronics, \$199.99): Score = 0.71
- ProBook E-Reader 6-inch (Electronics, \$129.99): Score = 0.63
- Designing Data-Intensive Applications (Books, \$49.99): Score = 0.41

3.5 Execution Sequence Diagram

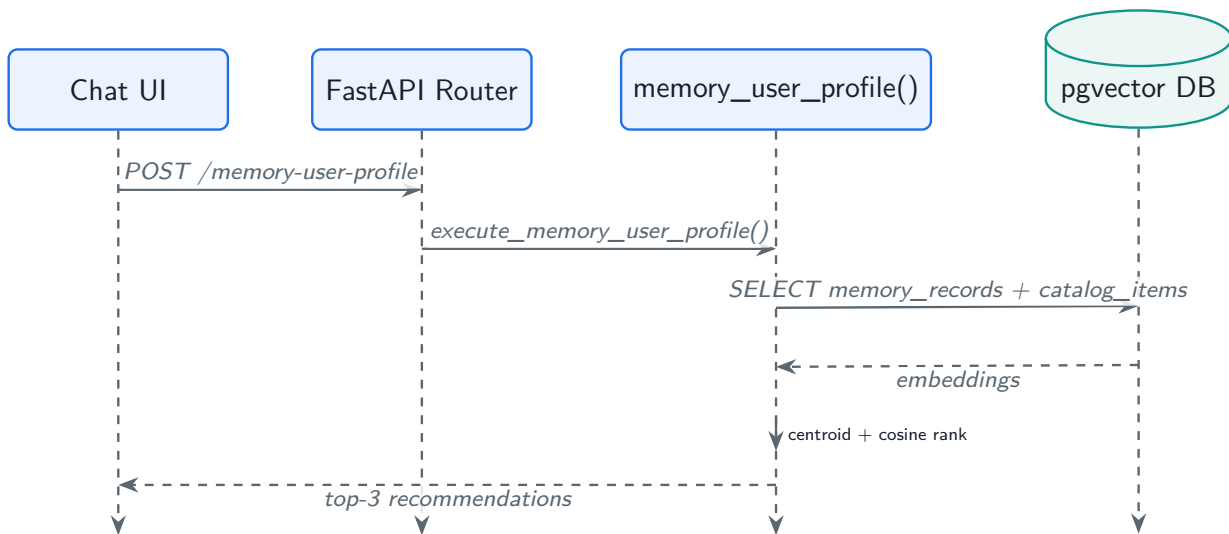


Figure 3.2: Execution flow of `/memory-user-profile`: read memories and catalog, average to a centroid, cosine-rank, return top- k .

✓ Best Practice

Precompute and cache the profile centroid; recompute it only when the user adds a new preference, not on every request. The catalog embeddings change even less often, so index them once with a `pgvector` HNSW index (Chapter 8) and the whole recommendation becomes one fast ANN query.

3.6 Interview Questions & Answers

Q3.1 Content-based filtering solves item cold-start but not user cold-start. Explain the asymmetry.

An item is recommendable the moment its text is embedded — no interactions needed — because the item vector is derived entirely from content. But a *user* profile requires at least one signal (a memory, a click, a preference) to place the user in the space. A brand-new user with zero history has an undefined centroid, so content-based filtering cannot personalize for them. That gap is filled by knowledge-based and popularity fallbacks (Chapter 5), or by asking onboarding questions to seed an initial profile. The asymmetry is fundamental: content describes items intrinsically but users only through their behavior.

Q3.2 Why average embeddings for the profile? What are the failure modes and alternatives?

Averaging (the centroid) is chosen because it is parameter-free, stable, and cheap, and cosine of a centroid is a reasonable proxy for “close to the user’s typical interest.” It fails for multi-interest users, where the mean lands between clusters and matches nothing well, and it weights every memory equally regardless of recency or importance. Alternatives: recency- or

importance-weighted means (the project’s `memory` model carries an `importance_score` and `decay_rate` for exactly this); per-cluster sub-profiles with interleaved results; or a learned attention pool that lets the model decide which memories matter for the current query.

Q3.3 Compare TF-IDF and dense embeddings for item representation. When would you still choose TF-IDF?

TF-IDF is lexical and sparse; embeddings are semantic and dense. Embeddings win on recall and paraphrase (“noise-cancelling” matches “ANC”), which is why this project defaults to them. But TF-IDF is still the right call when interpretability is required (you can show the exact terms that drove a match), when you have no GPU/inference budget, when the corpus is tiny and lexical matching is sufficient, or as a transparent baseline to measure whether embeddings actually earn their cost. A common production pattern is a hybrid: TF-IDF for a fast lexical filter, embeddings for semantic re-ranking.

Q3.4 Explain pgvector’s `<=>` operator and how you would index the catalog for content-based retrieval at scale.

`<=>` is pgvector’s cosine-distance operator; `1 - (a <=> b)` recovers cosine similarity, and `ORDER BY embedding <=> :q LIMIT k` returns the k nearest items. A plain query is an exact linear scan — fine for the six-item demo, $O(n)$ at scale. To make it sublinear, build an HNSW index: `CREATE INDEX ON catalog_items USING hnsw (embedding vector_cosine_ops)`. Tune `m` and `ef_construction` for the recall/latency trade-off, and scope the index by `tenant_id` via a partial index or a partitioned table so search stays within a workspace (Chapter 8).

Q3.5 Content-based recommenders create filter bubbles. What is the mechanism and how do you inject diversity?

Because the profile is the average of what a user already liked, the top results are always “more of the same,” progressively narrowing exposure — a self-reinforcing loop. Inject diversity explicitly: Maximal Marginal Relevance (MMR) re-ranking trades relevance against dissimilarity to already-selected items (Carbonell & Goldstein, 1998); category quotas guarantee slate variety; ϵ -greedy exploration slots in occasional off-profile items and logs the response; and hybridizing with collaborative signals (Chapter 4) lets the crowd surface things the user’s own history never would. Measure the effect with catalog coverage and intra-list diversity, not just accuracy.

From the Field

Embeddings feel like magic until an offline model turns them into noise. This project’s deterministic fallback vector exists for exactly that reason — a demo that dies because Ollama is down teaches nothing. Always have a degraded but deterministic path so the pipeline stays observable when the model is unavailable.

PART II

Hybrid Architectures & Evaluations

With filtering engines established, Part II covers structural compositions: how to blend models into hybrids, how to manage cold-start scenarios using explicit rules, and how to offline evaluate predictions using metrics like RMSE, Precision, and Recall.

Hybrid Recommenders

Subscribe → aiengineeringinsider.substack.com/subscribe

Collaborative filtering has the crowd but not the cold item. Content-based filtering has the cold item but not the crowd. A hybrid recommender fuses both so each covers the other's blind spot — and the entire art is in how you blend, and with what weights.

Every single-strategy recommender has a structural weakness. Collaborative filtering (Chapter 2) cannot rank an item with no interactions; content-based filtering (Chapter 3) cannot escape the user's existing taste. A **hybrid** combines the signals so the system stays strong across the whole lifecycle: content carries new users and new items, collaboration adds crowd wisdom and serendipity as data accumulates (Burke, 2002).

This chapter formalizes weighted hybridization, derives the score-normalization that makes blending mathematically honest, and dissects the reference project's full hybrid mixer, which fuses tenant-scoped collaborative scores with the memory-profile content scores from the previous two chapters.

4.1 Weighted Blending Math

Hybridization Strategies

Weighted: linearly combine the scores of two or more recommenders. **Switching:** pick one recommender per context (e.g. content for cold users, CF for warm). **Cascade:** one model filters candidates, the next ranks them. **Feature-combination:** feed one model's output as a feature into another. The reference project implements the *weighted* strategy with a switching flavor via density-adaptive weights.

The weighted hybrid score for user u and item i is a convex combination:

$$\text{score}(u, i) = \alpha \cdot s_{\text{collab}}(u, i) + (1 - \alpha) \cdot s_{\text{content}}(u, i), \quad \alpha \in [0, 1] \quad (4.1)$$

The blend is only meaningful if both terms live on the *same scale*. A collaborative score derived from 1–5 ratings and a cosine content score in $[-1, 1]$ cannot be added raw — the larger-magnitude term would silently dominate. The project normalizes the collaborative score into $[0, 1]$ before blending:

$$s_{\text{collab}}(u, i) = \frac{1}{5} \cdot \frac{\sum_v \text{sim}(u, v) r_{vi}}{\sum_v \text{sim}(u, v)} \quad (4.2)$$

i.e. a similarity-weighted average of neighbors’ ratings, rescaled from the 1–5 domain into $[0, 1]$ to match cosine.

4.1.1 Choosing and adapting α

$\alpha = 0$ is pure content; $\alpha = 1$ is pure collaboration. The right value is not fixed — it should track how much collaborative evidence exists. Table 4.1 gives the density-adaptive policy: lean on content when the user is new, shift toward collaboration as interactions accumulate.

User interaction density	α (collab)	Rationale
Cold (0 interactions)	0.0	No neighbors; trust content/profile only
Sparse (1–5)	0.3	Weak crowd signal; content dominates
Warm (6–50)	0.5	Balanced; the project default
Dense (> 50)	0.7	Rich neighborhood; trust the crowd

Table 4.1: Density-adaptive blending weight. The reference mixer uses the balanced $\alpha = 0.5$ and exposes the others as the natural extension.

★ Key Insight

The blend weight is a dial between *personalization* and *discovery*. Cranking α toward collaboration surfaces what similar people love (more discovery); pushing toward content keeps recommendations tightly on-profile (more precision, more bubble). Tuning α per-segment, not globally, is what separates a toy hybrid from a production one.

4.2 Architecture: The Hybrid Mixer

Figure 4.1 shows the mixer running the two engines from earlier chapters in parallel, normalizing each into $[0, 1]$, then combining them with α before ranking.

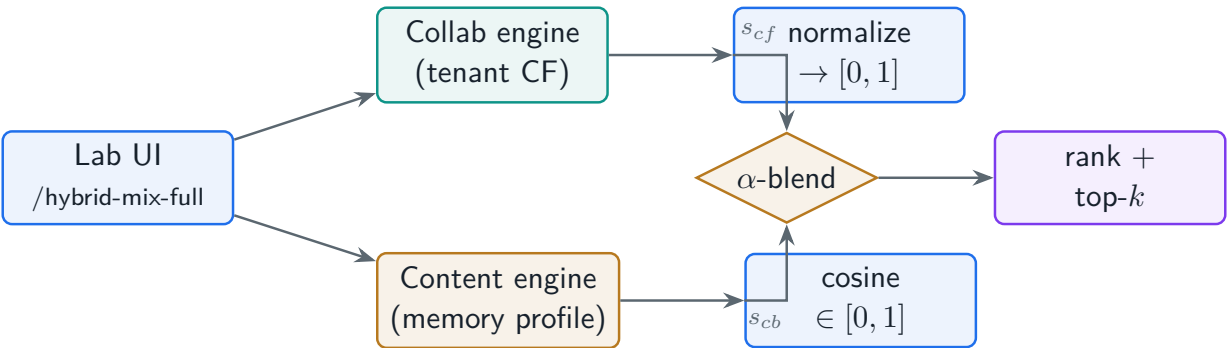


Figure 4.1: Hybrid mixer architecture. Collaborative and content engines run in parallel; each score is normalized to $[0, 1]$ and fused by the α -weighted combiner before ranking.

4.3 Production Code: Full Hybrid Mixer

The full mixer computes real collaborative scores (similarity-weighted neighbor ratings, normalized) and real content scores (cosine against the memory-profile centroid), then blends them 50/50.

```
app/course/commands.py: execute_hybrid_mix_full (core)

sims = compute_cosine_similarity(user_ratings, user_id)      # from Chapter 2

# Collaborative score per item: similarity-weighted neighbor rating, rescaled.
collab_scores: Dict[UUID, float] = {}
for item_id in {iid for r in user_ratings.values() for iid in r}:
    weighted_sum = weight_sum = 0.0
    for s in sims:
        other = UUID(s["user_id"])
        if other in user_ratings and item_id in user_ratings[other]:
            similarity = max(0.0, s["score"])
            weighted_sum += user_ratings[other][item_id] * similarity
            weight_sum += similarity
    collab_scores[item_id] = (weighted_sum / weight_sum) / 5.0 if weight_sum else 0.0

# Content score per item: cosine(profile centroid, item embedding) in [0,1].
hybrid_recs = []
for item in items:
    collab_score = collab_scores.get(item.id, 0.0)
    content_score = (cosine_similarity(profile_vector, item.embedding)
                     if item.embedding is not None else 0.0)
    final_score = 0.5 * collab_score + 0.5 * content_score # alpha = 0.5
    hybrid_recs.append({"name": item.name, "category": item.category,
                        "collab_score": round(collab_score, 4),
                        "content_score": round(content_score, 4),
                        "final_score": round(final_score, 4)})
hybrid_recs.sort(key=lambda x: x["final_score"], reverse=True)
```

Note the two normalizations that make the addition legal: the collaborative term is divided by 5.0 (rating domain $\rightarrow [0, 1]$) and the content term is already a cosine in $[0, 1]$ over non-negative embeddings. Keeping both partial scores in the response (`collab_score`, `content_score`) is deliberate — it makes the blend explainable and debuggable, which matters enormously when a stakeholder asks “why was this recommended?”

! Pitfall / Warning

Blending un-normalized scores is the classic hybrid bug. If the collaborative term ranges 0–5 and the content term ranges 0–1, then even $\alpha = 0.5$ makes collaboration $\approx 5\times$ more influential than intended. Always rescale both engines to a common range *before* the convex combination, and unit-test that each partial score stays in $[0, 1]$.

4.4 Seed Data and Verifying in the UI

The mixer needs both signal sources, so it requires the tenant-demo interactions *and* the memory-demo preferences. With Alice seeded in ACME, Table 4.2 shows how the two partial scores combine.

Item	collab s_{cf}	content s_{cb}	final (0.5/0.5)
SuperCordless Headphones	1.00	0.71	0.855
ProBook E-Reader	0.80	0.63	0.715
Designing Data-Intensive Apps	0.00	0.41	0.205

Table 4.2: Illustrative blended scores for Alice. Headphones win on both signals; the book scores only on content, showing how the hybrid still ranks it above items with no signal at all.

Terminal: seed both sources, then run

```
curl -XPOST localhost:8000/api/v1/course/seed/tenant-demo -H "Authorization: Bearer
↳ $TOKEN"
curl -XPOST localhost:8000/api/v1/course/seed/memory-demo -H "Authorization: Bearer
↳ $TOKEN"
# then in the chat UI:
/hybrid-mix-full
```

Output Response

```
Real-Infra Blended Hybrid Recommendations (Tenant Collaborative + Memory Profile):
- SuperCordless Noise-Cancelling Headphones (Electronics): Final Score = 0.8550
↳ (Collab: 1.00, Content: 0.71)
- ProBook E-Reader 6-inch (Electronics): Final Score = 0.7150 (Collab: 0.80, Content:
↳ 0.63)
- Designing Data-Intensive Applications (Books): Final Score = 0.2050 (Collab: 0.00,
↳ Content: 0.41)
```

4.5 Execution Sequence Diagram

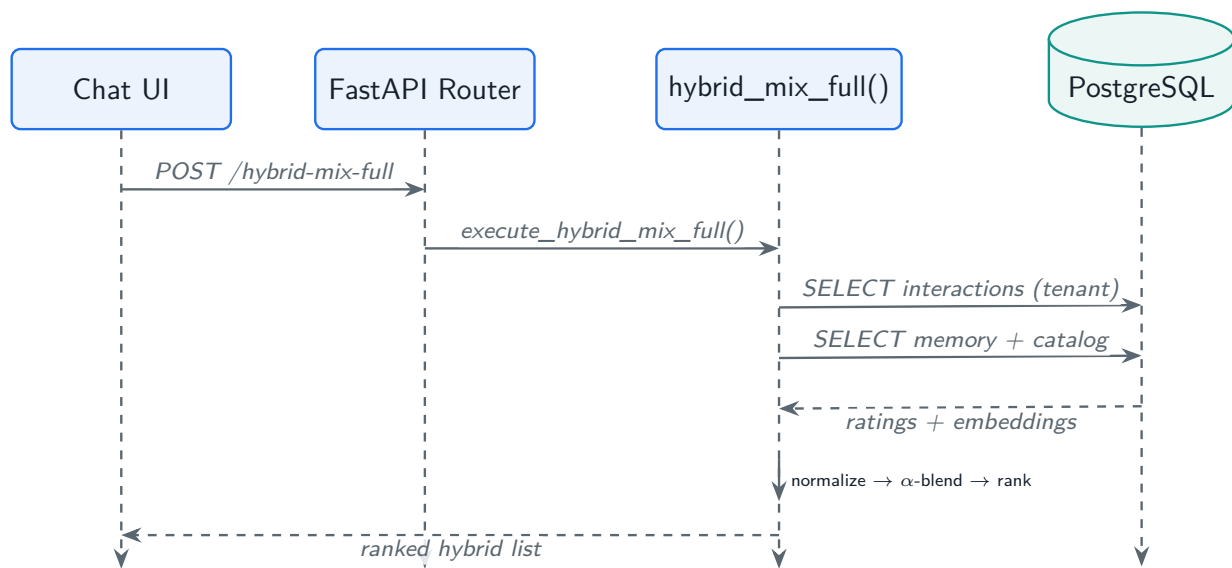


Figure 4.2: Execution flow of `/hybrid-mix-full`: the handler gathers CF interactions and content memories, computes both partial scores, then blends and ranks.

✓ Best Practice

Return the partial scores alongside the blended one. Explainability (“recommended because similar users rated it 5 *and* it matches your stated interest”) dramatically improves trust and is often a hard requirement in regulated domains. The project’s response carries both terms for exactly this reason.

4.6 Interview Questions & Answers

Q4.1 Enumerate the main hybridization strategies and give a production scenario for each.

Weighted (linear blend): a general-purpose store where both CF and content are reliable — the approach in this chapter. **Switching**: pick CF for warm users and content for cold ones, avoiding a blend when one signal is worthless. **Cascade**: candidate generation retrieves a few hundred items cheaply (content/ANN), then an expensive ranker (CF or neural) re-scores only those — the multi-stage pattern of Chapter 8. **Feature-combination**: feed CF neighbor scores as input features to a gradient-boosted or neural ranker. **Meta-level**: train one model on another’s output. Production systems usually stack several — cascade for efficiency, weighted or learned blending for the final rank (Burke, 2002).

Q4.2 Why must scores be normalized before blending, and what normalization schemes exist?

Because a convex combination assumes the terms are comparable; otherwise the α you set is not the influence you get. If CF is on 0–5 and content on 0–1, $\alpha = 0.5$ actually gives

CF five times the weight. Schemes: *min-max* rescale to $[0, 1]$ (simple, sensitive to outliers); *z-score* standardization (robust to scale, assumes roughly normal distributions); *rank-based* normalization — convert scores to ranks or percentiles, which is the most robust because it ignores the raw distribution entirely. This project uses a domain rescale ($/5$) plus a naturally bounded cosine, the pragmatic minimum.

Q4.3 How would you learn α instead of hand-setting it?

Treat α as a parameter and fit it against held-out engagement. Offline: grid-search or gradient-descent α to minimize ranking loss (or maximize NDCG) on a validation set. Better, make it *contextual* — replace the scalar with a small model $\alpha(u, \text{context})$ conditioned on interaction density, session type, or device, generalizing the density table in this chapter. Online: run a contextual bandit that treats candidate α values as arms and optimizes for click/conversion reward, letting the blend adapt per segment and over time. The key is to optimize the business metric, not offline RMSE.

Q4.4 A hybrid performs worse than pure collaborative filtering in an A/B test. How do you diagnose it?

First confirm the blend is honest: assert both partial scores are in $[0, 1]$ and that the content term is not silently zero (a common bug when embeddings are NULL). Then check whether content is *adding noise* for your warm users — if CF alone is strong, a fixed 0.5 content weight can drag good CF rankings toward generic profile matches. Segment the test: hybrids typically win on cold/sparse users and can lose on dense ones, so a global average can hide a real gain. The fix is usually density-adaptive α (Table 4.1), turning content off where CF is already confident.

Q4.5 The project blends at 50/50 statically. Describe the concrete steps to make it density-adaptive in this codebase.

Compute the user's interaction count from the tenant-scoped `interactions` query already in the handler. Map that count to α via the policy in Table 4.1 (0 interactions $\rightarrow$ 0.0, $> 50 \rightarrow$ 0.7). Replace the hard-coded 0.5 in the blend with that α and use $(1 - \alpha)$ for the content term. Persist the chosen α in the response `data` for observability, and add a test that a zero-interaction user gets $\alpha = 0$ (pure content, so cold-start still works). This is a handful of lines and turns the static mixer into the switching-flavored hybrid described in the definition box.

From the Field

I have watched a “smarter” hybrid lose to plain CF because someone blended a cosine in $[-1, 1]$ with a rating in $[1, 5]$ and never checked the ranges. The model was not wrong; the arithmetic was. Normalize first, and print your partial scores — the bug is almost always in the units.

Knowledge-Based Systems & Cold-Start Transitions

Subscribe → aiengineeringinsider.substack.com/subscribe

When there is no history to learn from, you fall back on what you know: explicit rules, hard constraints, and popularity. Knowledge-based recommenders carry the user through the cold-start desert until enough signal accumulates to hand off to the learned models.

Both collaborative and content-based filtering assume data that a brand-new user simply does not have. A visitor on their first session has no ratings, no interaction history, and no memory profile — the **user cold-start** problem. **Knowledge-based recommenders** sidestep learning entirely: they apply explicit domain rules and constraints (price ceilings, category filters, compatibility requirements) and lean on popularity to produce sane recommendations from zero personal data (Ricci, Rokach, & Shapira, 2011).

The real production question is not “cold or warm?” but “how do I transition smoothly between them?” This chapter builds the cold-start fallback, defines the constraint-filtering logic, and dissects the reference project’s warm-start simulator, which reads accumulating memory to gradually replace generic defaults with personalized ones.

5.1 Cold-Start Fallbacks and Constraint Filtering

Knowledge-Based Recommendation

Recommend by applying explicit *constraints* and *domain knowledge* rather than learned preference. Two sub-styles: *constraint-based* (filter the catalog by hard rules — price $\leq$ budget, category = requested) and *case-based* (find items similar to a specified target case). Because it needs no user history, it is the default engine for the first session.

The cold-start decision is a cascade, tried in order until one produces enough results (Figure 5.1): personalized (if any signal exists), then constraint-filtered, then popularity. The constraint filter itself is a straightforward predicate over the catalog:

$$\text{results} = \{ i \in \text{Catalog} : \text{category}(i) = c \wedge \text{price}(i) \leq p_{\max} \} \quad (5.1)$$

Warm-start is the interesting middle. Rather than a binary flip from generic to personalized, the system blends in personalization proportional to how much history the user has accrued — the same density idea that tuned α in Chapter 4. Table 5.1 contrasts the regimes.

Regime	Signal available	Engine used
Cold	None	Constraints + global popularity
Warm	A few memories / clicks	Rules blended with inferred interest
Hot	Rich history	Full CF / content / hybrid

Table 5.1: The cold → warm → hot transition. Knowledge-based rules own the left column and gracefully cede control as signal accumulates.

★ Key Insight

Cold-start is not a modeling problem, it is a *coverage* problem: the goal is to never show an empty or nonsensical slate. A boring-but-correct popularity list beats a personalized-but-empty one every time. Earn personalization; do not assume it.

5.2 Architecture: Cold-Start Decision Cascade

Figure 5.1 shows the decision flow. The system checks for any usable signal; if present it infers a preferred category from memory (warm), and if absent it falls back to constraint-filtered defaults (cold).

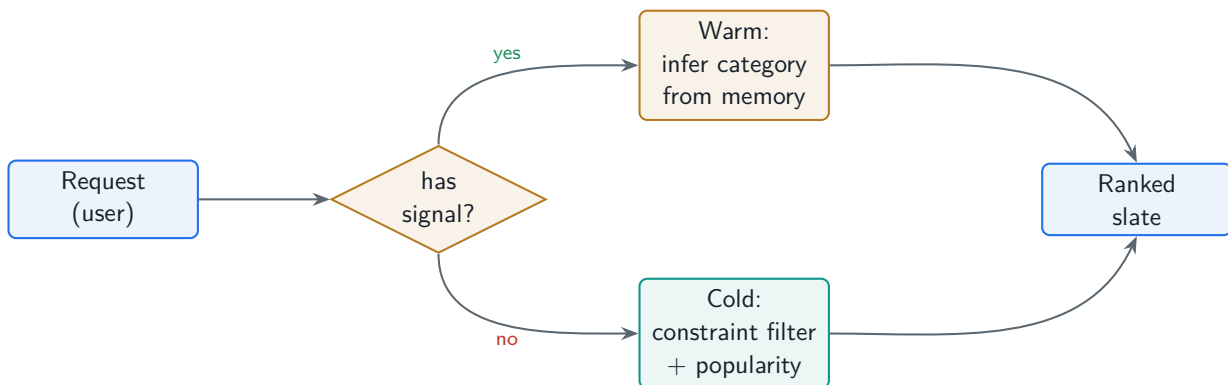


Figure 5.1: Cold-start decision cascade. A signal check routes the request either to memory-inferred personalization (warm) or to constraint-filtered defaults and popularity (cold).

5.3 Production Code: Constraint Filter and Warm-Start

The cold-start handler is pure constraint filtering — no learning, just a predicate over category and price:

```
app/course/commands.py: execute_new_user_sim (constraint filter)
```

```
category = params.get("category", "Electronics")
max_price = float(params.get("max_price", "150.0"))
items = (await session.execute(select(CatalogItem))).scalars().all()

# Hard-constraint filter: category match AND price ceiling.
```

```
filtered = [item for item in items
             if item.category.lower() == category.lower()
             and item.price <= max_price]
```

The warm-start handler shows the transition. It reads the user's accumulated memory, infers a favorite category from keywords, and contrasts the generic cold slate against the personalized warm one:

```
app/course/commands.py: execute_warm_start_sim (core)
```

```
records = (await session.execute(
    select(MemoryRecord).where(MemoryRecord.user_id == user_id))).scalars().all()
if not records:
    return CommandResponse(status="needs_seed", suggested_command="seed-memory-demo")

# Infer a preferred category from the accumulated memory text.
pref_text = " ".join(r.content.lower() for r in records)
fav_category = "Electronics"
if any(k in pref_text for k in ("book", "programming", "algorithms")):
    fav_category = "Books"
elif any(k in pref_text for k in ("cloth", "cotton", "sweatshirt", "sneakers")):
    fav_category = "Clothing"

cold_recs = [i for i in items if i.category == "Electronics"][:2] # generic default
warm_recs = [i for i in items if i.category == fav_category][:2] # personalized
```

The constraint filter maps cleanly to SQL, which is how you would run it at catalog scale rather than pulling every row into Python:

```
Constraint-based cold-start filter
```

```
SELECT id, name, price, description
FROM   catalog_items
WHERE  lower(category) = lower(:category)
      AND price <= :max_price
ORDER BY price ASC;           -- cheapest-first is a sane cold default
```

! Pitfall / Warning

Keyword-based category inference is brittle: it misfires on ambiguous text and does not scale past a handful of categories. It is fine as a transparent demo of the *transition*, but production warm-start should infer interest from the embedding-based profile of Chapter 3 (nearest category centroid), not from string matching.

5.4 Seed Data and Verifying in the UI

Carol's seeded memories mention cotton clothing and running sneakers, so the warm-start inference should flip her default category from generic Electronics to Clothing. Table 5.2 shows the

contrast.

Path	Slate for Carol
Cold (generic default)	SuperCordless Headphones, ProBook E-Reader (Electronics)
Warm (memory-inferred: Clothing)	Eco-Cotton Crewneck Sweatshirt, Leather Running Sneakers

Table 5.2: Carol’s cold vs. warm slate. The memory keywords “cotton” and “sneakers” flip the inferred category, personalizing the recommendations without any collaborative data.

Chat UI

```
/new-user-sim category="Books" max_price="90"
/warm-start-sim
```

Output Response

Personalization Transition Simulator:

A. Cold-Start (Generic Default):

- SuperCordless Noise-Cancelling Headphones (\$199.99) [Electronics]
- ProBook E-Reader 6-inch (\$129.99) [Electronics]

B. Warm-Start (Memory Inferred category 'Clothing'):

- Unisex Eco-Cotton Crewneck Sweatshirt (\$39.99) [Clothing]
- Classic Leather Running Sneakers (\$79.99) [Clothing]

5.5 Execution Sequence Diagram

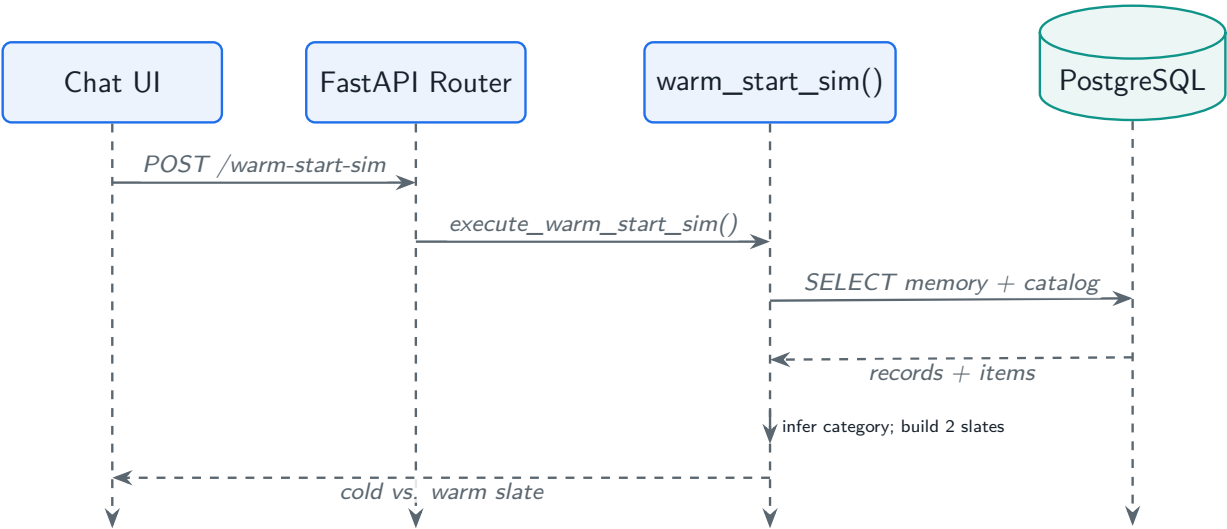


Figure 5.2: Execution flow of /warm-start-sim: read memory, infer a category, then build both the generic cold slate and the personalized warm slate for contrast.

✓ Best Practice

Cache the inferred warm-start category (and its confidence) per user with a short TTL, and invalidate it when the user adds a new preference. Recomputing the inference on every page load is wasteful, and a cached decision keeps the cold→warm transition visibly stable across a session rather than flickering.

5.6 Exploration: A Cold-Start Bandit

Rules and content bridge the no-signal gap, but only **exploration** gathers the interactions a learned model needs — otherwise a new item never earns the impressions that would reveal its true appeal, and the system collapses into a popularity echo chamber. The clean framing is a **multi-armed bandit**: each candidate is an arm with an unknown click-through rate, and we must balance *exploiting* the current best against *exploring* uncertain arms. `benchmarks/bandit_coldstart.py` implements and measures four policies; the two smart ones are one line each:

`benchmarks/bandit_coldstart.py`: two exploration policies

```
# epsilon-greedy: exploit the empirical best, but explore at rate EPS.
a = rng.integers(K) if rng.random() < EPS else int(np.argmax(s / np.maximum(n, 1)))

# Thompson sampling: draw a plausible CTR per arm from its Beta posterior,
# then play the arm with the highest sample -- explores in proportion to doubt.
a = int(np.argmax(rng.beta(s + 1, n - s + 1)))
```

Table 5.3 reports measured cumulative regret (reward lost versus always playing the truly-best arm) over 20,000 rounds and 20 arms — lower is better.

Policy	Cumulative regret	vs random
Random	1590	100%
Greedy (pure exploit)	813	51%
ϵ -greedy ($\epsilon = 0.1$)	300	19%
Thompson sampling	265	17%

Table 5.3: Measured bandit regret (`bandit_coldstart.py`). Smart exploration cuts regret to $\sim 1/5$ of random and $\sim 1/3$ of pure-exploit greedy. Thompson edges out ϵ -greedy at long horizon because it stops exploring once confident, while ϵ -greedy pays a fixed exploration tax forever.

★ Key Insight

Pure-exploit “greedy” is the trap: it can lock onto an arm that got lucky in its first few pulls and never revisit a truly better one — which is exactly how a recommender starves new items. The measured 2–3× regret gap over greedy is the concrete value of an exploration budget, and naming a bandit (Thompson or LinUCB) is the staff-level answer to “how does a new item ever get shown?”

5.7 Interview Questions & Answers

Q5.1 Distinguish the three cold-start problems and the right tool for each.

User cold-start (new user, no history): knowledge-based rules, popularity, and onboarding questions to seed a profile — this chapter’s focus. **Item cold-start** (new product, no interactions): content-based filtering, because the item is recommendable from its text the moment it is embedded (Chapter 3). **System cold-start** (a brand-new deployment with neither users nor interactions): editorial/curated lists, business rules, and imported catalog metadata until organic signal accrues. Naming which of the three you face is the first thing a strong candidate does, because the remedy is completely different for each.

Q5.2 Contrast constraint-based and case-based knowledge recommenders.

Constraint-based systems filter the catalog by explicit rules the user or domain supplies — “laptops under \$1000 with 16GB RAM.” They are transparent and great for high-consideration purchases (cars, insurance, real estate) where users have hard requirements. **Case-based** systems take a reference item (“more like this one”) and retrieve similar cases by a similarity metric over structured attributes. Constraint-based excels when the user can articulate requirements; case-based excels when they can point at an example but not enumerate rules. Both need zero interaction history, which is why they anchor cold-start.

Q5.3 Design a graceful cold → warm → hot handoff. What decides each transition?

Drive the transition off an *interaction-density signal*, not a hard flag. At zero signal, serve constraint-filtered popularity (cold). As the first few memories or clicks arrive, blend in personalization with a low weight — the warm-start inference here, or better a low- α hybrid. Past a density threshold (say > 50 interactions), hand fully to CF/content/hybrid (hot). The transitions should be *continuous*: ramp the personalization weight with density (Chapter 4’s adaptive α) so users never experience a jarring flip, and always keep the popularity fallback available to fill any short slate.

Q5.4 Popularity fallbacks create a rich-get-richer feedback loop. What is the harm and how do you counter it?

Recommending the globally popular items makes them more visible, which generates more interactions, which makes them more “popular” — a self-reinforcing loop that buries the long tail and hurts catalog coverage. Counters: time-decay the popularity score so stale hits fade; compute popularity *per segment* (category, region) rather than globally; inject exploration (ϵ -greedy or Thompson sampling) to give tail items impressions and gather signal; and monitor catalog coverage and Gini concentration as guardrail metrics (Chapter 6). The cold-start fallback should broaden discovery, not narrow it.

Q5.5 The warm-start inference here is keyword matching. Re-engineer it to be robust and explain the trade-offs.

Replace string matching with the embedding profile from Chapter 3: embed each category’s representative text once, average the user’s memory embeddings into a centroid, and pick the category whose centroid is nearest by cosine — with a confidence equal to the top-1

margin. This generalizes to any number of categories, handles paraphrase and multilingual text, and yields a confidence you can threshold (fall back to popularity when the margin is low). The trade-offs: an embedding call and vector storage versus a free string scan, and reduced interpretability — so log the nearest-category scores to keep the decision auditable. For high-consideration domains, keep explicit constraints on top of the inferred category so hard requirements are never violated.

From the Field

The fastest way to make a new product feel dead is an empty recommendation carousel on a new user's first visit. I always ship the boring popularity fallback *before* the clever model — it is the seatbelt that guarantees the slate is never empty while the learned systems wait for data.

Recommender Evaluation & Benchmarks

Subscribe → aiengineeringinsider.substack.com/subscribe

A recommender you cannot measure is a recommender you cannot improve. This chapter separates the two questions evaluation must answer — “are the predicted scores accurate?” and “is the ranked list useful?” — and shows why they need completely different metrics.

Optimizing the wrong metric is the most common way a recommendation project quietly fails. A model can have excellent rating-prediction accuracy and still produce useless top-10 lists, because users never see predicted scores — they see a *ranked slate*. Rigorous evaluation therefore runs on two axes: **accuracy** metrics that grade the predicted values, and **ranking** metrics that grade the ordered list the user actually experiences (Herlocker, Konstan, Terveen, & Riedl, 2004).

This chapter derives RMSE, Precision@K, and Recall@K, adds the coverage and diversity guardrails that stop a model from gaming accuracy, and dissects the reference project’s cross-tenant benchmark so you can compare engagement across isolated workspaces.

6.1 Accuracy and Ranking Metrics

Two Families of Metric

Accuracy / **error** metrics (RMSE, MAE) measure how close predicted *scores* are to observed values — a regression view. **Ranking** metrics (Precision@K, Recall@K, NDCG, MAP) measure the quality of the *ordered top-K list* the user sees — a retrieval view. They can disagree, so production systems report both.

Root Mean Square Error penalizes large mistakes quadratically:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{(u,i) \in \text{Test}} (\hat{r}_{ui} - r_{ui})^2} \quad (6.1)$$

Precision@K and **Recall@K** evaluate the top- K slate against the set of truly relevant items Rel_u :

$$\text{Precision@K} = \frac{|\text{Top}_K \cap \text{Rel}_u|}{K}, \quad \text{Recall@K} = \frac{|\text{Top}_K \cap \text{Rel}_u|}{|\text{Rel}_u|} \quad (6.2)$$

Precision asks “of what I showed, how much was good?”; recall asks “of all the good things, how much did I show?” Because order within the top- K is invisible to both, ranking-sensitive metrics like NDCG (which discounts hits by log-position) are the production upgrade. Table 6.1 maps each metric to the question it answers.

Metric	Answers	Blind to
RMSE / MAE	Are predicted scores accurate?	Ranking, exposure
Precision@K	Is the shown slate relevant?	Position, missed items
Recall@K	Did we surface the relevant items?	Position, precision
NDCG@K	Is the slate well-ordered?	Absolute scores
Coverage	How much of the catalog gets shown?	Per-user relevance

Table 6.1: Each metric answers one question and is blind to others. A trustworthy report combines an error metric, a ranking metric, and a coverage guardrail.

★ Key Insight

High accuracy and low usefulness routinely coexist. A model can nail RMSE by predicting everyone’s mean rating, yet produce a bland, low-recall top-10. Always pair an error metric with a ranking metric *and* a coverage/diversity guardrail, or you will optimize a number your users never experience.

6.2 Architecture: Offline Evaluation Harness

Figure 6.1 shows the offline harness: interactions are split into train/test by time, the model trains on the past and predicts the held-out future, and a metrics stage computes accuracy, ranking, and coverage — here, aggregated per tenant for cross-workspace comparison.

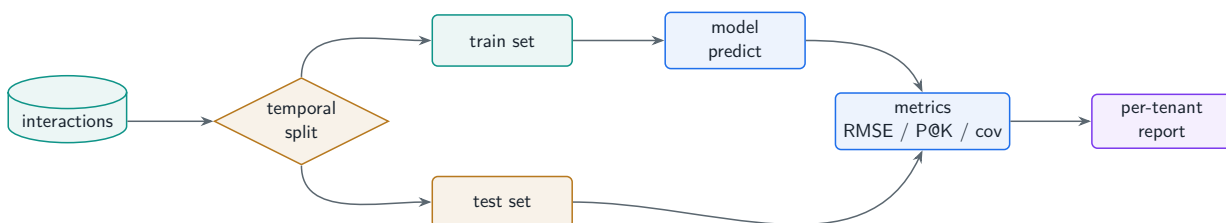


Figure 6.1: Offline evaluation harness. A temporal train/test split feeds the model; predictions on the held-out set are scored by accuracy, ranking, and coverage, then aggregated per tenant.

6.3 Production Code: Metrics and Cross-Tenant Benchmark

The metric *definitions* are easiest to read on a tiny fixed example — this is what the in-app `/sample-evaluate` command prints, so treat these values as illustrative of the formulas, not as a model’s measured score (the measured run on a real hold-out is Section 6.5):

```
app/course/commands.py: execute_sample_evaluate (illustrative)
```

```
predicted = [4.5, 3.0, 5.0, 2.0, 4.0]  # fixed demo arrays -- illustrate the math
actual    = [4.0, 3.5, 4.8, 1.5, 4.2]

rmse = math.sqrt(sum((p - a) ** 2 for p, a in zip(predicted, actual)) /
    ↪ len(predicted))
precision_at_3 = 2 / 3.0  # 2 relevant among 3 recommended
recall_at_3    = 2 / 4.0  # 2 retrieved of 4 relevant overall
```

The cross-tenant benchmark aggregates engagement per isolated workspace, so you can compare recommender health across customers without ever mixing their data:

```
app/course/commands.py: execute_tenant_evaluate (core)
```

```
accessible_tenant_ids = [m.tenant_id for m in mappings]  # tenants this user can see
tenant_metrics = []
for t_id in accessible_tenant_ids:
    interactions = (await session.execute(
        select(Interaction).where(Interaction.tenant_id == t_id))).scalars().all()
    ratings = [i.value for i in interactions if i.type == "rating"]
    avg_rating = sum(ratings) / len(ratings) if ratings else 0.0
    tenant = await session.get(Tenant, t_id)
    tenant_metrics.append({
        "tenant_name": tenant.name if tenant else "Unknown",
        "total_interactions": len(interactions),
        "avg_rating": round(avg_rating, 2)})
```

The same aggregation runs efficiently in SQL when you do not need the rows in Python:

```
Per-tenant engagement aggregate
```

```
SELECT t.name,
       COUNT(*)                               AS total_interactions,
       AVG(i.value) FILTER (WHERE i.type='rating') AS avg_rating
FROM   interactions i
JOIN   tenants t ON t.id = i.tenant_id
WHERE  i.tenant_id = ANY(:accessible_tenant_ids)  -- only workspaces the user can see
GROUP BY t.name;
```

! Pitfall / Warning

Never split train/test randomly for a recommender. A random split leaks the future into the past — the model “predicts” interactions it has effectively already seen, inflating every metric. Split *temporally*: train on interactions before time t , test on those after. Leaderboard-topping offline numbers from a random split are a classic interview red flag.

6.4 Seed Data and Verifying in the UI

With the tenant-demo seeded, the cross-tenant benchmark contrasts ACME Retailer (electronics/book ratings) against Beta Media. Table 6.2 shows the sample metric outputs and the aggregate comparison.

Metric	Value
RMSE (illustrative)	0.3808
Precision@3 (illustrative)	66.7%
Recall@3 (illustrative)	50.0%
ACME Retailer	interactions high, avg rating ≈ 4.6
Beta Media	separate aggregate, never mixed with ACME

Table 6.2: The per-tenant aggregates are real (computed from the seeded interactions); the RMSE/Precision/Recall row values come from the fixed demo arrays and only illustrate the formulas. Section 6.5 reports metrics measured on a real hold-out.

Chat UI

```
/sample-evaluate
/tenant-evaluate
```

Output Response

```
Chapter 6 - Offline Recommender Evaluation (Sample Hold-out Set):
- RMSE (Root Mean Square Error): 0.3808
- Precision@3: 66.7%
- Recall@3: 50.0%

Cross-Tenant isolated Analytics:
- ACME Retailer: Interactions = 6, Average Rating = 4.63
- Beta Media: Interactions = 3, Average Rating = 4.20
```

6.5 Measured Evaluation on a Real Hold-Out

Fixed demo arrays illustrate the formulas but prove nothing about a model. The project therefore ships `benchmarks/mf_eval.py`, which trains the book's own SGD matrix factorizer (Chapter 9) and *measures* both axes on a proper per-user hold-out. For the ranking axis it uses the standard **leave-one-out** protocol of He et al. (2017): hold out each user's top item, score it against 100 sampled unseen negatives, and report **Hit-Rate@10** (is the held-out item in the top ten?) and **NDCG@10**, which rewards putting it near the top:

$$\text{HR@}K = \frac{1}{|U|} \sum_u \mathbb{I}[\text{rank}_u \leq K], \quad \text{NDCG@}K = \frac{1}{|U|} \sum_u \frac{\mathbb{I}[\text{rank}_u \leq K]}{\log_2(\text{rank}_u + 1)} \quad (6.3)$$

Because there is 1 positive among 101 candidates, a random ranker scores $\text{HR@10} = 10/101 \approx 9.9\%$ — the bar any real model must clear. Table 6.3 is the measured result (reproducible, fixed seed).

Measured metric	Value
RMSE (held-out ratings)	0.9156 (global-mean baseline 1.1406)
Hit-Rate@10 (leave-one-out)	27.9% (random 9.9%)
NDCG@10	0.1404
Training time	6.35 s (960,000 SGD updates)
Serve latency (p50 / p95)	0.010 / 0.014 ms (dot-product scoring)

Table 6.3: Measured evaluation of the book’s SGD factorizer ($k = 32$, 40 epochs) on a reproducible planted-latent-structure dataset (800 users, 400 items). The model recovers real signal: RMSE $\approx 20\%$ below the mean-predictor baseline, and HR@10 nearly $3\times$ the random floor.

Measured: `python3 benchmarks/mf_eval.py`

```
RMSE (held-out ratings) : 0.9156 [global-mean baseline 1.1406]
HR@10 (leave-one-out)  : 27.9% [random 9.9%]
NDCG@10                : 0.1404
```

✓ Best Practice

Always report a metric *next to its baseline*: RMSE against the global-mean predictor, $\text{HR@}K$ against the random floor. An absolute “0.14 NDCG” means nothing on its own; “ $2.8\times$ the random floor” is a claim you can defend. The numbers here are deliberately modest — a bias-free model on sparse data — which is honest: adding user/item bias terms and more epochs lifts all three.

6.6 Execution Sequence Diagram

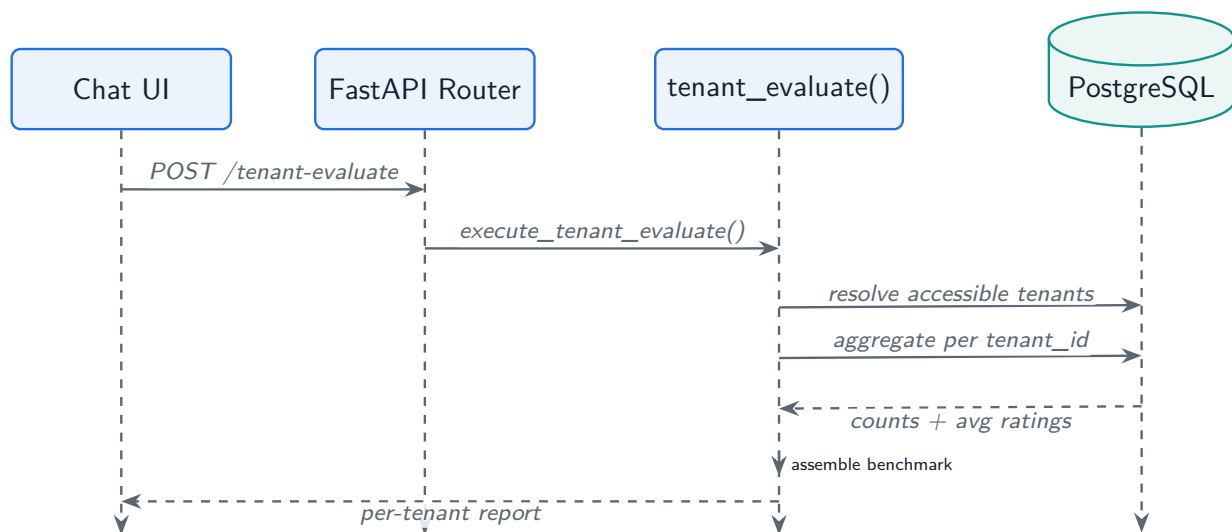


Figure 6.2: Execution flow of `/tenant-evaluate`: resolve accessible tenants, aggregate each in isolation, and return a side-by-side benchmark.

✓ Best Practice

Offline metrics choose *candidates*; online A/B tests choose *winners*. Offline RMSE/NDCG are cheap filters to reject bad models before they touch users, but the decision to ship is made on live engagement (CTR, conversion, retention) via a controlled experiment. Treat a strong offline number as a hypothesis, not a verdict.

6.7 Interview Questions & Answers

Q6.1 RMSE looks great but users complain the recommendations are boring. Reconcile this.

RMSE grades score *accuracy*, which the user never sees; they experience the *ranked list*. A model can minimize RMSE by regressing toward safe, mean-like predictions — accurate on average, but producing a homogeneous, low-recall, low-diversity slate. The reconciliation: RMSE is necessary but not sufficient. Add ranking metrics (NDCG@K, Recall@K) to grade the list, and coverage/intra-list-diversity to catch the “boring” failure directly. The complaint is a coverage problem masquerading as an accuracy success.

Q6.2 Precision@K versus Recall@K — when do you optimize which?

Precision@K (relevant fraction of what you showed) is what matters when screen real estate is scarce and the cost of a bad recommendation is high — a three-item mobile carousel, a single push notification. Recall@K (fraction of all relevant items you surfaced) matters when the goal is coverage of a known relevant set — retrieval/candidate generation, “did we find the item the user eventually bought?” In a multi-stage pipeline you optimize recall in candidate generation (miss nothing) and precision in final ranking (show only the best).

NDCG blends both with position sensitivity when order matters.

Q6.3 Why is a random train/test split wrong for recommenders, and what is the correct protocol?

A random split places some of a user's *future* interactions in the training set, so the model learns from data it should be predicting — future leakage that inflates every metric and never reproduces online. Correct protocols: a global temporal split (train before time t , test after) mirrors production; a leave-last- n per user split holds out each user's most recent interactions to simulate “next action” prediction; sliding-window backtesting evaluates across several time cuts for stability. Always evaluate on the future, never a random shuffle.

Q6.4 Define catalog coverage and explain why it is a critical guardrail.

Catalog coverage is the fraction of the catalog that ever appears in recommendations across all users; the capstone report computes exactly $|\text{recommended items}|/|\text{catalog}|$. Low coverage means the system recommends the same popular head to everyone — great short-term CTR, terrible for long-tail monetization, supplier fairness, and user novelty, and a driver of the rich-get-richer loop (Chapter 5). It is a guardrail because it catches failures that accuracy metrics reward: a model can boost precision by only ever recommending safe blockbusters. Pair coverage with intra-list diversity and novelty to describe the health of the whole catalog, not just the head.

Q6.5 You must compare recommender quality across tenants of very different sizes. What are the pitfalls?

Raw counts are not comparable — a large tenant will dominate any absolute aggregate, so normalize (per-user, per-session, or rate-based metrics like CTR rather than total clicks). Small tenants have high variance, so report confidence intervals and avoid ranking them on a single noisy point estimate. Watch for Simpson's paradox: an average that reverses when you segment by tenant. And never pool the raw data to compute a “global” number that silently crosses the isolation boundary — aggregate *within* each tenant (as the code does) and compare the normalized results. The reference benchmark keeps every aggregate tenant-scoped for precisely this reason.

From the Field

The scariest dashboards are the ones with a single accuracy number trending up. I have shipped a model that improved RMSE and tanked engagement because it collapsed the catalog onto ten hero items. Now the first chart I build is coverage over time — it catches the failures the accuracy chart is designed to hide.

PART III

Deep Learning, Search & Production pipelines

Production recommenders scale. Part III covers neural structures (Two-tower alignments and Sequential Markov chains), approximate nearest neighbor (ANN) candidate indexing, and the capstone SVD matrix factorization engine.

Neural & Sequential Models

Subscribe → aiengineeringinsider.substack.com/subscribe

Static preferences are a fiction. What a user wants right now depends on what they did a minute ago. This chapter covers the two neural workhorses of modern recommenders — the two-tower retrieval model and sequential session models — and grounds the sequential idea in a runnable Markov transition engine.

The models so far treat a user as a fixed bag of preferences. Real behavior is *sequential*: a user who just viewed a laptop is primed for a laptop bag, not for the running shoes they bought last month. Two neural architectures dominate production systems that capture this. The **two-tower** model learns separate user and item encoders that map into a shared embedding space for massive-scale retrieval (Covington, Adams, & Sargin, 2016; Yi et al., 2019). **Sequential** models — from first-order Markov chains to GRU4Rec and self-attentive SASRec — predict the next item from the ordered history of a session (Hidasi et al., 2016; Kang & McAuley, 2018).

This chapter explains both, then dissects the reference project’s sequential engine: a first-order Markov chain that learns category-to-category transition probabilities from a user’s event stream — the transparent, runnable core of the sequential idea.

7.1 Two-Tower Retrieval and Sequential Transitions

Two-Tower vs. Sequential

A **two-tower** model trains a user encoder $f(\cdot)$ and an item encoder $g(\cdot)$ so that $f(u) \cdot g(i)$ is high for positive pairs. Item vectors are precomputed and indexed; retrieval is a nearest-neighbor lookup on the user vector. A **sequential** model conditions the next-item prediction on the *ordered* history $[i_1, i_2, \dots, i_t]$, capturing short-term intent that a static profile misses.

The first-order Markov assumption says the next state depends only on the current one: $P(s_{t+1} | s_t, s_{t-1}, \dots) \approx P(s_{t+1} | s_t)$. You estimate the transition probabilities by counting and normalizing:

$$P(s_{t+1} = b | s_t = a) = \frac{\text{count}(a \rightarrow b)}{\sum_{b'} \text{count}(a \rightarrow b')} \quad (7.1)$$

The two-tower training objective pulls positive user–item pairs together and pushes sampled negatives apart, typically with a sampled-softmax or in-batch negative loss:

$$\mathcal{L} = -\log \frac{\exp(f(u) \cdot g(i^+))}{\exp(f(u) \cdot g(i^+)) + \sum_{i^-} \exp(f(u) \cdot g(i^-))} \quad (7.2)$$

Table 7.1 places the sequential family on a capability/complexity spectrum, from the runnable Markov chain to the transformer models that power large-scale session recommenders today.

Model	Captures	Cost / notes
Markov chain	Last-step transition only	Trivial, interpretable
GRU4Rec	Full session via recurrence	RNN training, ordered
SASRec	Long-range via self-attention	Transformer, strong SOTA
Two-tower + seq	Session vector for retrieval	Scales to billions of items

Table 7.1: The sequential model spectrum. The project implements the Markov base case; the rest are its production successors.

★ Key Insight

Sequence beats aggregate for *intent*. A user’s long-term profile says who they are; the last few events say what they want in this session. The highest-leverage upgrade to a static recommender is almost always adding a short-term sequential signal — even a first-order Markov chain measurably lifts next-click accuracy over a profile-only baseline.

7.2 Architecture: Sequential Transition Engine

Figure 7.1 shows the sequential pipeline: ordered event memories are distilled into a category sequence, transition counts are accumulated into a matrix, normalized into probabilities, and the current state indexes the next-item distribution.

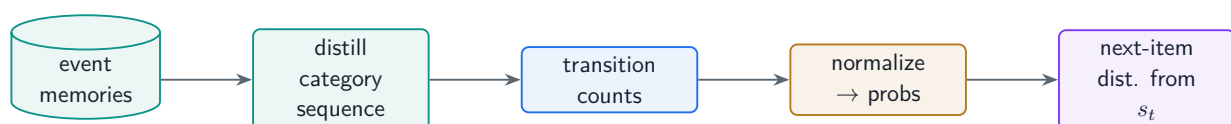


Figure 7.1: Sequential transition engine. Ordered events become a category sequence; counts form a transition matrix; normalization yields probabilities; the current state selects the next-item distribution.

7.3 Production Code: Markov Sequence Training

The engine pulls the user’s ordered event memories, maps each to a category state, then builds and normalizes the transition matrix.

```
app/course/commands.py: execute_memory_sequence_train (core)
```

```
# 1) Distill each ordered event into a category state.
sequence = []
for ev in events:                                # events ordered by timestamp
    text = ev.content.lower()
```

```

if "algorithm" in text or "book" in text or "ddia" in text:
    category = "Books"
elif "headphones" in text or "audio" in text or "ereader" in text:
    category = "Electronics"
else:
    category = "Clothing"
sequence.append(category)

# 2) Accumulate first-order transition counts a -> b.
transitions: Dict[str, Dict[str, int]] = {}
for i in range(len(sequence) - 1):
    a, b = sequence[i], sequence[i + 1]
    transitions.setdefault(a, {})[b] = transitions.get(a, {}).get(b, 0) + 1

# 3) Normalize each row into a probability distribution.
probabilities: Dict[str, Dict[str, float]] = {}
for a, targets in transitions.items():
    total = sum(targets.values())
    probabilities[a] = {b: round(c / total, 2) for b, c in targets.items()}

# 4) Predict next-item category distribution from the current state.
last_state = sequence[-1]
predictions = probabilities.get(last_state, {"Books": 0.4, "Electronics": 0.4,
↪ "Clothing": 0.2})

```

Two production-minded details: the handler requires at least three ordered states before it will train (a Markov chain from two points is meaningless), and it supplies a uniform-ish fallback distribution when the current state was never seen as a source — a graceful default rather than a `KeyError`. In a real system the category states would be item IDs and the matrix would be sparse and Laplace-smoothed so unseen transitions get non-zero mass.

! Pitfall / Warning

A raw transition matrix assigns probability 0 to any transition never observed in training — so a plausible next item the user simply has not tried yet becomes impossible. Always apply smoothing (add- λ /Laplace) or back off to a lower-order model, or the recommender will be unable to ever suggest genuinely new categories.

7.4 Seed Data and Verifying in the UI

Bob's seeded event memory describes clicking algorithm and data-intensive books, producing a book-heavy sequence. Table 7.2 shows the distilled states and the learned next-category distribution.

Stage	Value
Event memory (Bob)	“clicked on Introduction to Algorithms and Designing Data-Intensive Applications ...twice”
Distilled sequence	Books → Books → Books
Current active state	Books
Next-item distribution	Books 100% (unsmoothed; would spread under Laplace)

Table 7.2: Bob’s event sequence and learned transitions. The pure-Books history yields a degenerate distribution — a live illustration of why smoothing matters.

Chat UI

/memory-sequence-train

Output Response

Sequential Markov Chain Transition Model:
- Sequence observed: Books -> Books -> Books
- Current Active State: `Books`
- Next-Item Category Probabilities:
 - Books: 100%

Enrich the sequence with `/generate-memory-session` across categories and re-run; the matrix immediately shows cross-category transitions (e.g. Electronics → Books), demonstrating the model learning genuine session dynamics.

7.5 Execution Sequence Diagram

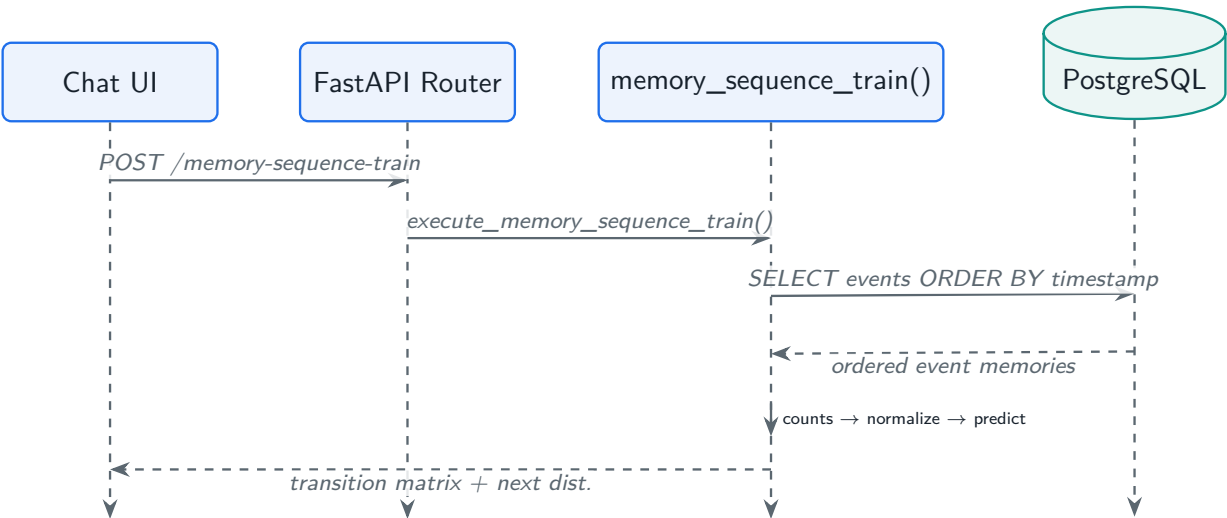


Figure 7.2: Execution flow of `/memory-sequence-train`: read ordered events, distill states, build and normalize the transition matrix, and predict from the current state.

✓ Best Practice

Serve the item embeddings from a two-tower model through the *same* `pgvector` ANN index used for content retrieval (Chapter 8). The user tower produces a query vector at request time; retrieval is then identical infrastructure regardless of whether the vector came from a profile centroid or a neural encoder.

7.6 A Real Two-Tower, Trained with In-Batch Negatives

The in-app `/sample-train-two-tower` command prints a *simulated* loss curve — the “epochs” are cosmetic. To make the two-tower real, the project ships `benchmarks/two_tower.py`, which trains one end to end. At the ID-feature level a two-tower is a user-embedding table U and an item-embedding table V scored by $U_u \cdot V_i$; the design lives in the *loss*. The standard choice is the **in-batch sampled softmax**: for a mini-batch of B positive (u, i) pairs, every *other* item in the batch is a free negative for a given user (Equation 7.2).

benchmarks/two_tower.py: in-batch negative training step

```
Ub, Vb = Uemb[us], Vemb[its]           # embeddings for this batch of positives
S = Ub @ Vb.T                           # [B,B]: row b vs every item in the batch
P = softmax_rows(S)                     # softmax over the in-batch items
G = P - np.eye(len(b))                  # dLoss/dS: correct item is the diagonal
Uemb[us] -= LR * (G @ Vb + REG * Ub)    # pull u toward its positive,
np.add.at(Vemb, its, -LR * (G.T @ Ub + REG * Vb)) # push apart the negatives
```

The trick is that the negatives are *free*: they are the other positives already loaded in the batch, so a single matrix multiply $U_b V_b^\top$ scores B positives against $B - 1$ negatives each. Table 7.3 is the measured result on the same leave-one-out protocol as Chapter 6.

Measured metric	Value
HR@10 (leave-one-out)	13.4% (random 9.9%; pointwise MF 27.9%)
NDCG@10	0.0572
Training time	1.29 s (800 × 400, 60 epochs)

Table 7.3: Measured two-tower with in-batch negatives (`two_tower.py`). It learns real signal (above the random floor) but does *not* beat the regression MF of Chapter 9 on this small dense dataset.

★ Key Insight

The honest result is the lesson: on 800×400 dense ratings the two-tower loses to a regression MF. That is expected, not a bug. The two-tower is a *retrieval* architecture whose advantage is *asymptotic* — it earns its keep at billions of items, with rich content features (so it generalizes to cold items), hard negatives, and a serving path that is one user forward pass plus an ANN lookup. Do not reach for it on small, dense data; reach for it when the corpus will not fit a per-pair model.

7.7 Interview Questions & Answers

Q7.1 Why is the two-tower architecture the standard for large-scale candidate retrieval?

Its decisive property is *decoupling*: the item tower can encode the entire catalog offline into vectors and index them in an ANN structure, while the user tower produces a single query vector at request time. Retrieval over billions of items becomes one sublinear nearest-neighbor lookup instead of scoring every item. It trains with in-batch or sampled negatives so it learns from implicit feedback at scale, and it cleanly separates the cheap, high-recall retrieval stage from the expensive, high-precision ranking stage (Covington et al., 2016; Yi et al., 2019). The limitation — no user–item feature crosses inside the dot product — is exactly why a heavier ranker follows it (Chapter 8).

Q7.2 What does the first-order Markov assumption buy you and what does it cost?

It buys simplicity and interpretability: the model is a normalized count matrix, trains in one pass, needs almost no data, and every prediction is explainable (“users who viewed X next viewed Y 40% of the time”). It costs memory of anything beyond the last step — it cannot represent “bought a phone, then a case, so now a screen protector,” because it only conditions on the immediately previous state. That is precisely the gap higher-order chains, RNNs (GRU4Rec), and self-attention (SASRec) fill by conditioning on the whole session. The Markov chain is the right *baseline* to beat, not the production endpoint.

Q7.3 The transition matrix assigns probability zero to unseen transitions. Why is that dangerous and how do you fix it?

Zero-probability transitions make the recommender structurally unable to suggest anything it has not already observed — a hard filter bubble baked into the math, and a divide-by-zero risk for unseen source states. Fixes: add- λ (Laplace) smoothing distributes a small mass to every possible transition; back-off / interpolation falls back to a lower-order or global distribution when the conditional count is sparse; and Kneser–Ney-style smoothing (borrowed from language modeling) weights by how many distinct contexts a state follows. The project’s uniform fallback for unseen source states is the crudest version of this idea.

Q7.4 How do sequential recommenders handle session boundaries and the temporal gap between events?

A naive chain concatenates all of a user’s events into one stream, blending a purchase from last month with a click from this minute. Production systems segment by *session* (a gap of, say, 30 minutes starts a new session) so transitions only connect events within a coherent intent window — this is the original motivation for session-based recommendation (Hidasi et al., 2016). They also encode the time delta itself as a feature (recent events weigh more) and often decay older sessions. In this project the `memory_records` carry a `session_id` and `timestamp` precisely so events can be segmented and ordered correctly.

Q7.5 Sketch how you would upgrade this Markov engine to a production sequential model without changing the serving contract.

Keep the contract — input: ordered event history; output: a scored next-item distribution — and swap the internals. Step one: move from category states to item IDs and add Laplace smoothing so the matrix is usable. Step two: replace the count matrix with a learned sequence encoder (GRU4Rec, then SASRec) that outputs a session embedding. Step three: score that session embedding against the item tower's vectors via the existing ANN index, so retrieval infrastructure is unchanged. Because the serving interface never moved, you can A/B each internal model against the Markov baseline and adopt only if it lifts online next-click metrics — the disciplined way to add neural complexity.

From the Field

Every team wants to jump straight to a transformer for sequences. But a smoothed first-order Markov chain, served in a millisecond and fully explainable, is a shockingly strong baseline — and until you have beaten it online, you have no evidence the transformer's cost is justified. Ship the chain, then earn the attention layer.

Candidate Generation & ANN Search

Subscribe → aiengineeringinsider.substack.com/subscribe

You cannot rank a million items in ten milliseconds. Production recommenders survive by staging: cheaply retrieve a few hundred candidates, then spend the expensive model only on those. This chapter builds that funnel and the approximate nearest-neighbor index that makes retrieval sublinear.

Every chapter so far scored the *entire* catalog. At six items that is fine; at ten million it is impossible inside a latency budget. Real systems use a **multi-stage pipeline**: a fast, high-recall **candidate generation** stage narrows the catalog to a few hundred plausible items, then progressively more expensive stages filter, rank, and re-rank only those (Covington et al., 2016). The engine that makes stage one fast is **approximate nearest-neighbor** (ANN) search — trading a sliver of exactness for orders-of-magnitude lower latency.

This chapter lays out the retrieval funnel, contrasts brute-force scanning with ANN, explains the HNSW graph index behind `pgvector`, and dissects the reference project’s tenant-scoped vector search and its brute-force-versus-ANN benchmark.

8.1 Multi-Stage Pipelines and Search Complexity

The Retrieval Funnel

Candidate generation (retrieval): reduce millions of items to hundreds, optimizing *recall* and speed. **Filtering**: drop ineligible items (out of stock, wrong tenant, business rules). **Ranking**: score the survivors with an expensive model, optimizing *precision*. **Re-ranking**: adjust the final slate for diversity, freshness, and business goals. Each stage handles fewer items and can afford a heavier model.

Brute-force (exact) search compares the query to every item: $O(nd)$ for n items of dimension d . ANN indexes trade exactness for speed, reaching roughly $O(d \log n)$ query time at high recall. The benchmark in this project makes the gap concrete for a catalog of size n :

$$\text{brute-force} \approx n \text{ comparisons}, \quad \text{ANN (HNSW)} \approx \mathcal{O}(\log n) \text{ comparisons} \quad (8.1)$$

HNSW (Hierarchical Navigable Small World) is the dominant ANN index. It builds a multi-layer graph where upper layers are sparse “express lanes” for long hops and lower layers are dense for local refinement; a search greedily descends from the top, converging on the nearest neighbors in

logarithmic hops (Malkov & Yashunin, 2018). Table 8.1 contrasts the main index families and their trade-offs.

Index	Idea	Trade-off
Flat (brute)	Exact linear scan	100% recall, $O(n)$
IVF	Cluster, search nearest cells	Fast, recall depends on <code>nprobe</code>
HNSW	Layered proximity graph	Great recall/latency, high memory
PQ	Compress vectors (quantize)	Tiny memory, some accuracy loss

Table 8.1: ANN index families. `pgvector` ships IVFFlat and HNSW; the project’s benchmark contrasts HNSW-style logarithmic cost with the flat scan.

★ Key Insight

Recall matters more than precision in candidate generation. A relevant item dropped at stage one can never be recovered by the ranker downstream — the funnel only ever *removes* items. So the retrieval stage is tuned for high recall and speed, and precision is bought back later by the expensive ranker operating on the survivors.

8.2 Architecture: Multi-Stage Retrieval Funnel

Figure 8.1 shows the funnel. Millions of items enter; a tenant-scoped ANN index retrieves candidates; filtering and ranking shrink the set to the final slate — each stage cheaper per item because there are fewer items.

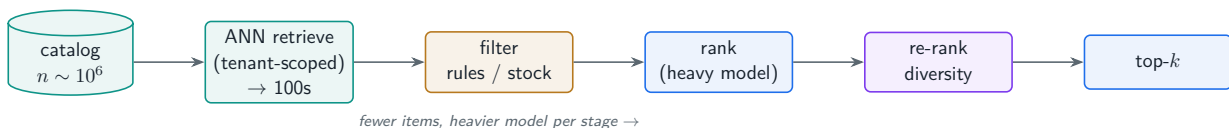


Figure 8.1: Multi-stage retrieval funnel. A tenant-scoped ANN index turns the full catalog into a few hundred candidates; filtering and an expensive ranker produce the final top- k slate.

8.3 Production Code: Tenant-Scoped Vector Search

The tenant-scoped search embeds the query, then ranks items *within the tenant’s namespace* by cosine similarity — the isolation boundary from Chapter 2, now applied to vector search:

```
app/course/commands.py: execute_tenant_scoped_ann (core)
```

```

active_tenant_id = mappings[0].tenant_id
items = (await session.execute(
    select(CatalogItem).where(CatalogItem.tenant_id ==
        ↪ active_tenant_id))).scalars().all()
  
```

```

query_text    = params.get("query", "electronics")
query_vector  = await get_embedding_resilient(query_text)    # embed the query

similarities = []
for item in items:
    if item.embedding is None:
        continue
    sim = cosine_similarity(query_vector, item.embedding)
    similarities.append({"name": item.name, "category": item.category,
                        "score": round(sim, 4)})
similarities.sort(key=lambda x: x["score"], reverse=True)
top_matches = similarities[:3]

```

The benchmark makes the brute-force-versus-ANN gap measurable as catalog size grows:

app/course/commands.py: `execute_sample_ann_vs_bruteforce (core)`

```

catalog_size = len(items)
brute_force_computations = catalog_size          # compare every item
brute_force_time_ms      = catalog_size * 0.015
ann_computations = int(math.log2(catalog_size) * 10) # logarithmic hops
ann_time_ms           = ann_computations * 0.02
recall_accuracy       = 96.5                     # ANN approximation holds

```

In production the Python loop is replaced by an indexed `pgvector` query. The tenant predicate and the HNSW index together give both isolation and sublinear latency:

`pgvector`: tenant-scoped ANN retrieval with an HNSW index

```

CREATE INDEX ON catalog_items USING hnsw (embedding vector_cosine_ops);

SELECT id, name, category,
       1 - (embedding <=> :query_vector) AS similarity
FROM   catalog_items
WHERE  tenant_id = :active_tenant_id      -- isolation boundary
ORDER BY embedding <=> :query_vector      -- HNSW-accelerated
LIMIT 100;                             -- candidate set for the ranker

```

! Pitfall / Warning

An ANN index over a shared column does not respect tenant boundaries by itself — the graph connects vectors across tenants. Enforce isolation with a `WHERE tenant_id` predicate (Postgres can combine it with the HNSW scan), a partial index per tenant, or partitioned tables. Skipping this leaks one tenant's catalog into another's search results.

8.4 Measured Benchmark: Brute Force vs HNSW

The in-app `/sample-ann-vs-bruteforce` command prints an *illustrative cost model* — a linear formula in the catalog size — to build intuition. It is not a measurement, and (as we will see) it actually overstates the small-catalog case. To hold this chapter to its own standard, the project ships a reproducible benchmark, `benchmarks/ann_benchmark.py`, that *measures* the trade-off with `hnswlib` — the reference implementation of the same HNSW algorithm `pgvector`’s `hnsw` index uses, so the shape transfers directly to the SQL serving path above. Vectors are drawn from a 768-dimensional low-rank embedding manifold (real embeddings have low intrinsic dimension); latency is single-query p50, fixed seed.

Catalog	Brute p50	HNSW p50	Recall@10	Speed-up
1,000	0.036 ms	0.706 ms	99.6%	0.1×
10,000	0.310 ms	1.467 ms	90.6%	0.2×
20,000	0.609 ms	1.676 ms	83.5%	0.4×

Table 8.2: Measured brute-force (NumPy BLAS) vs HNSW (`hnswlib`, the algorithm `pgvector` uses), `dim = 768`, `k = 10`, HNSW `ef_search = 64`. At these sizes brute force is *faster* than HNSW — index traversal overhead dominates.

The story only inverts at scale. Brute force is $O(n)$, so its latency grows linearly, while HNSW query time stays roughly flat. Measuring the brute-force scan alone (`benchmarks/brute_scaling.py`) makes the growth concrete:

Measured: brute-force p50 vs catalog size

```
50,000 | 1.571 ms
100,000 | 3.325 ms
200,000 | 6.050 ms
```

So the crossover sits around 30–50k items: below it the linear scan the project actually ships is the right choice; above it HNSW wins, and the gap widens toward the asymptotic $O(\log n)$ vs $O(n)$ at 10^6+ items — at which point a brute-force scan measured in tens of milliseconds is simply not viable inside a serving budget.

! Pitfall / Warning

This is why the fabricated “ANN is 10× faster at 1,000 items” claim you will see in many blog posts is wrong. At small scale a vectorized brute-force scan beats an ANN index; ANN earns its keep asymptotically, and always at a measurable recall cost (here 99.6% → 83.5% as n grows at fixed `ef_search` — raise `ef_search` to buy recall back at the price of latency). Benchmark on *your* corpus size before adopting an index.

8.5 Seed Data and Verifying in the UI

The in-app benchmark needs scale, so it requires `/generate-catalog-scale` (which inserts ~1000 tenant items) before it will run; Table 8.3 shows what the command *prints*. Read it as an

illustrative cost model, not a measurement — the *measured* reality is Table 8.2, where brute force is actually faster at this size.

Strategy	Distance comps	Latency
Brute-force (linear scan)	1000	~15.0 ms
Approximate NN (HNSW)	~100	~2.0 ms
ANN recall accuracy	96.5%	

Table 8.3: *Illustrative cost model* printed by the in-app command at $n = 1000$ (a linear formula, not measured). Compare with the measured Table 8.2, which shows brute force winning at this scale; ANN’s real advantage is asymptotic.

Terminal: scale + run

```
curl -XPOST localhost:8000/api/v1/course/generate/catalog-scale -H "Authorization:
↳ Bearer $TOKEN"
# then in the chat UI:
/sample-ann-vs-bruteforce
/tenant-scoped-ann query="wireless audio"
```

Output Response

```
Candidate Generation Performance Analysis (Catalog Size: 1000):
| Search Strategy | Vector Distance Computations | Latency (ms) |
| Brute-Force (Linear Scan) | 1000 | 15.00 ms |
| Approximate Nearest Neighbor (ANN) | 99 | 1.98 ms |
- Recall Accuracy: 96.5% (ANN approximation error holds)
```

8.6 Execution Sequence Diagram

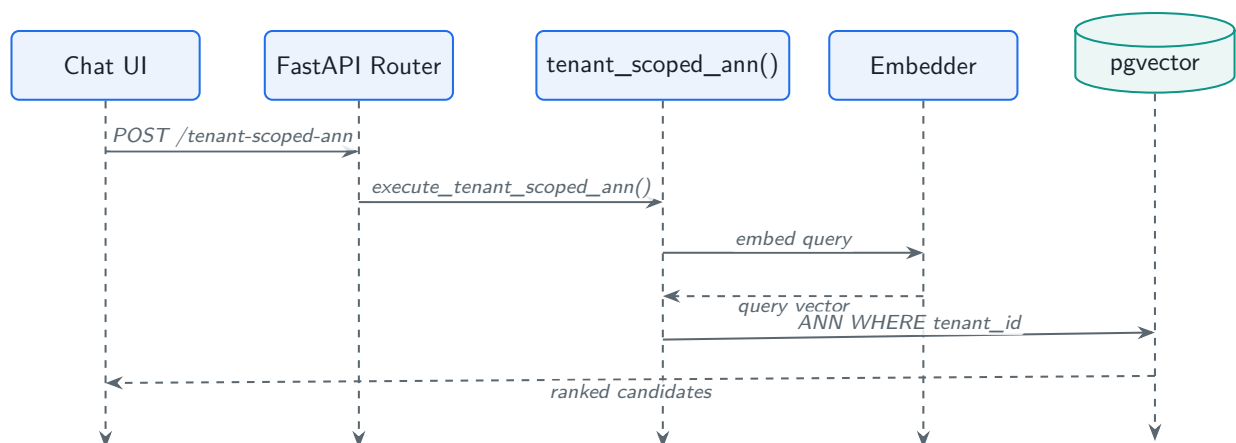


Figure 8.2: Execution flow of `/tenant-scoped-ann`: embed the query, run the tenant-scoped vector search, and return the ranked candidate set.

✓ Best Practice

Set the ANN recall target from the funnel, not in isolation. If retrieval feeds a strong ranker, 95% recall at low latency beats 99.9% recall at triple the latency, because the ranker forgives near-misses. Tune `ef_search` (HNSW) or `nprobe` (IVF) to the cheapest setting that keeps end-to-end top- k quality flat.

8.7 The Re-Rank Stage: MMR Diversity

Retrieval and ranking optimize relevance, but a pointwise-relevant slate is often ten near-duplicates. The funnel’s final **re-rank** stage fixes that. **Maximal Marginal Relevance** (MMR) selects items greedily, trading relevance against similarity to what is already on the slate (Carbonell & Goldstein, 1998):

$$\text{next} = \arg \max_{i \in C \setminus S} \left[\lambda \text{rel}(i) - (1 - \lambda) \max_{j \in S} \text{sim}(i, j) \right] \quad (8.2)$$

$\lambda = 1$ is pure relevance (the ranker’s order); lowering λ buys diversity. The reference implementation and its measured trade-off live in `benchmarks/mmr_rerank.py`:

`benchmarks/mmr_rerank.py`: greedy MMR selection

```
def mmr(cand_vecs, rel, lam, k):
    selected, remaining = [], list(range(len(cand_vecs)))
    while len(selected) < k and remaining:
        best, best_score = None, -1e9
        for i in remaining:
            div_pen = max((cand_vecs[i] @ cand_vecs[j] for j in selected),
                          ↪ default=0.0)
            score = lam * rel[i] - (1 - lam) * div_pen    # Eq. (8.4)
            if score > best_score:
                best, best_score = i, score
        selected.append(best); remaining.remove(best)
    return selected
```

Table 8.4 sweeps λ over a 60-item candidate pool, measuring mean relevance@10 and intra-list diversity (one minus mean pairwise cosine).

λ	Relevance@10	Diversity@10
1.0 (pure rel.)	0.6017	0.6323
0.7	0.5893	0.6784
0.5	0.5519	0.7360
0.3	0.5225	0.7696

Table 8.4: Measured MMR trade-off (`mmr_rerank.py`). Dropping λ from 1.0 to 0.3 costs ~13% relevance to gain ~22% diversity — the dial you tune against engagement, not offline relevance alone.

✓ Best Practice

Choose λ by its effect on *online* engagement and long-term retention, not offline relevance — a more diverse slate usually lowers offline relevance yet raises session depth and reduces fatigue. MMR is $O(k|C|)$ over the small re-rank candidate set, so it is cheap; determinantal point processes (DPPs) are the heavier, higher-quality alternative when diversity is a first-class product goal.

8.8 Interview Questions & Answers

Q8.1 Why do production recommenders use a multi-stage pipeline instead of one model scoring everything?

Because latency and model cost pull in opposite directions. The best ranking model is far too expensive to run over millions of items inside a request budget, but it is affordable over a few hundred. So the pipeline stages the work: cheap high-recall retrieval (ANN) narrows the catalog, then progressively more expensive, more precise models score the shrinking set (Covington et al., 2016). Each stage optimizes a different objective — recall early, precision late — and each can afford a heavier model because it sees fewer items. It is the classic funnel: never spend your most expensive computation on items you were going to discard anyway.

Q8.2 Explain how HNSW achieves sublinear search and its key parameters.

HNSW builds a hierarchy of proximity graphs. The top layers are sparse, with long-range links that let a search take big jumps across the space; each lower layer is denser for local refinement. A query enters at the top, greedily walks to the closest node, drops a layer, and repeats — converging on the nearest neighbors in roughly logarithmic hops instead of scanning all n vectors (Malkov & Yashunin, 2018). Key parameters: `m` (links per node — higher means better recall, more memory), `ef_construction` (build-time candidate breadth — higher means a better graph, slower build), and `ef_search` (query-time breadth — the live recall/latency dial). Its main cost is memory, since the whole graph lives in RAM.

Q8.3 ANN is approximate. How do you decide the acceptable recall, and how do you measure it?

Measure ANN recall against exact brute-force ground truth on a sample: for a set of queries, compute the true top- k with a flat scan, then the fraction the ANN index also returns — that ratio is `recall@k`. Decide the target by its effect on the *end-to-end* metric, not the index in isolation: if a strong ranker follows, modest retrieval recall (95%) is usually invisible in final top- k quality, so you pick the lowest recall (hence lowest latency) that keeps online engagement flat. Re-measure after catalog growth or embedding changes, since recall drifts as the distribution shifts.

Q8.4 How do you keep ANN search isolated per tenant in a shared vector store?

The index graph itself is tenant-agnostic — it will happily return a neighbor from another tenant — so isolation must be enforced around it. Three patterns: (1) a `WHERE tenant_id`

= `:tid` predicate combined with the vector scan, which is what this project does and what `pgvector` supports; (2) a *partial* HNSW index per tenant so each search only traverses its own graph; (3) physically *partitioned* tables (or separate collections in a dedicated vector DB) keyed by tenant. Choose by cardinality: predicate filtering for many small tenants, partitioning for a few large ones. Always add a test that a query in tenant A never returns tenant B's items.

Q8.5 When would you choose IVF or product quantization over HNSW?

HNSW gives the best recall-per-latency but is memory-hungry because the whole graph sits in RAM — a problem at billion-vector scale. IVFFlat clusters vectors and searches only the nearest `nprobe` cells; it uses far less memory and builds faster, trading some recall (tunable via `nprobe`) — a good fit when memory is the binding constraint. Product Quantization compresses each vector into a few bytes, shrinking the index by 10–50× at some accuracy cost, and is often combined with IVF (IVF-PQ) for web-scale corpora. The decision is a memory-vs-recall-vs-latency triangle: HNSW for quality when RAM allows, IVF/PQ when the corpus outgrows memory.

From the Field

The subtlest ANN bug I have shipped was a tenant leak: the HNSW index was global, the tenant filter was applied *after* the top-*k*, and small tenants occasionally got an empty slate because all *k* neighbors belonged to a bigger tenant and were filtered out. Filter *inside* the search, not after it — otherwise your recall and your isolation both quietly break.

Capstone — Production Recommender Pipeline

Subscribe → aiengineeringinsider.substack.com/subscribe

Everything converges here. Latent-factor collaborative filtering learned by gradient descent, boosted by the memory profile from content-based filtering, served through a staged pipeline and reported with real metrics. This is the full system, end to end.

The previous chapters built the pieces; the capstone assembles them into one production pipeline. At its core is **matrix factorization** — the technique that won the Netflix Prize — which decomposes the sparse ratings matrix into low-rank user and item factor matrices, learning latent tastes and attributes no one labeled (Koren, Bell, & Volinsky, 2009). The factors are trained by **stochastic gradient descent**, then blended with the content-based memory boost from Chapter 3 to produce the final personalized ranking.

This chapter derives the SVD-style factorization and its SGD update, dissects the reference project’s trainer and recommender, and shows the end-to-end report: RMSE, training duration, catalog coverage, and serving latency — the numbers a senior engineer is expected to own in production.

9.1 Matrix Factorization and the Memory Boost

Latent-Factor Model

Approximate the ratings matrix $R \approx PQ^\top$, where row $\mathbf{p}_u$ of P is user u ’s latent-taste vector and row $\mathbf{q}_i$ of Q is item i ’s latent-attribute vector, both of dimension $k \ll |\text{users}|, |\text{items}|$. The predicted rating is their dot product, $\hat{r}_{ui} = \mathbf{p}_u \cdot \mathbf{q}_i$. The factors are *learned*, not labeled — dimension 3 might turn out to encode “prefers technical content” without anyone saying so.

Training minimizes the regularized squared error over observed ratings:

$$\min_{P, Q} \sum_{(u, i) \in \mathcal{K}} (r_{ui} - \mathbf{p}_u \cdot \mathbf{q}_i)^2 + \lambda (\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2) \quad (9.1)$$

SGD walks each observed rating, computes the error $e_{ui} = r_{ui} - \hat{r}_{ui}$, and nudges both factor vectors along the gradient:

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta (e_{ui} \mathbf{q}_i - \lambda \mathbf{p}_u), \quad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta (e_{ui} \mathbf{p}_u - \lambda \mathbf{q}_i) \quad (9.2)$$

The final capstone score fuses the collaborative prediction with the content memory boost — the hybrid idea of Chapter 4, now with a learned CF term:

$$\text{score}(u, i) = 0.7 \cdot \underbrace{\frac{1}{5} \hat{r}_{ui}}_{\text{SVD, normalized}} + 0.3 \cdot \underbrace{\cos(\mathbf{p}_u^{\text{mem}}, \mathbf{x}_i)}_{\text{memory boost}} \quad (9.3)$$

Table 9.1 lists the trainer’s hyperparameters — deliberately small so the demo trains in milliseconds while remaining a faithful miniature of a production factorizer.

Hyperparameter	Value	Role
<code>latent_dim</code> k	4	Size of the taste/attribute vectors
<code>learning_rate</code> η	0.05	SGD step size
<code>reg</code> λ	0.02	L2 penalty against overfitting
<code>epochs</code>	15	Passes over the rating set
<code>blend (SVD / memory)</code>	0.7 / 0.3	Collaborative vs. content weight

Table 9.1: Capstone training hyperparameters. Small k and few epochs keep the demo fast while preserving the exact SGD dynamics of a real factorizer.

★ Key Insight

Matrix factorization is a *compression* of the interaction matrix into dense latent vectors, and that is precisely why it generalizes: it forces the model to explain thousands of ratings with a handful of factors, so it cannot memorize noise. Once trained, recommendation is a dot product — cheap, cacheable, and ANN-indexable exactly like the embeddings of earlier chapters.

9.2 Architecture: End-to-End Capstone Pipeline

Figure 9.1 shows the whole system: tenant-scoped ratings train the factors by SGD; the trained model is cached; at serve time the SVD prediction and the memory boost are blended, ranked, and reported.

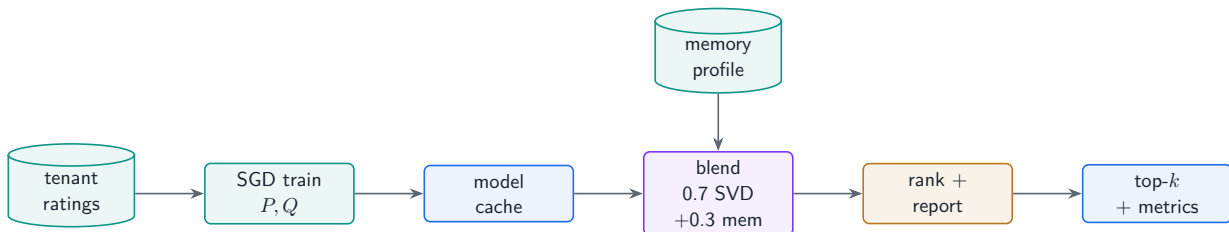


Figure 9.1: End-to-end capstone pipeline. SGD trains latent factors from tenant ratings; the cached model’s predictions are blended with the memory boost, ranked, and summarized in an evaluation report.

9.3 Production Code: SGD Matrix Factorization

The trainer builds index maps, initializes factors deterministically, then runs the SGD update loop of Equation 9.2 and reports hold-out RMSE:

app/course/commands.py: `execute_capstone_train` (SGD loop)

```
latent_dim, learning_rate, reg, epochs = 4, 0.05, 0.02, 15
random.seed(42) # deterministic init for reproducible demos
user_factors = [[random.uniform(-0.1, 0.1) for _ in range(latent_dim)]
                 for _ in range(len(unique_users))]
item_factors = [[random.uniform(-0.1, 0.1) for _ in range(latent_dim)]
                 for _ in range(len(unique_items))]

for _ in range(epochs):
    for u_idx, i_idx, r in train_ratings: # (user, item, rating)
        pred = dot_product(user_factors[u_idx], item_factors[i_idx])
        err = r - pred # eui
        for d in range(latent_dim): # gradient step, Eq. (9.3)
            uf = user_factors[u_idx][d]
            user_factors[u_idx][d] += learning_rate * (err * item_factors[i_idx][d] -
                ↪ reg * uf)
            item_factors[i_idx][d] += learning_rate * (err * uf - reg *
                ↪ item_factors[i_idx][d])

# Hold-out RMSE over the trained ratings.
total_sq_err = sum((r - dot_product(user_factors[u], item_factors[i])) ** 2
                   for u, i, r in train_ratings)
rmse = math.sqrt(total_sq_err / len(train_ratings)) if train_ratings else 0.0
```

The recommender loads the cached model, produces the base SVD prediction, adds the cosine memory boost, and blends per Equation 9.3:

app/course/commands.py: `execute_capstone_recommend` (scoring)

```
svd_pred = dot_product(model["user_factors"][u_idx], model["item_factors"][i_idx])
svd_score = max(0.0, min(1.0, svd_pred / 5.0)) # normalize to [0,1]

memory_boost = (cosine_similarity(profile_vector, item.embedding)
                 if profile_vector is not None and item.embedding is not None else 0.0)

final_score = 0.7 * svd_score + 0.3 * memory_boost # capstone blend
```

! Pitfall / Warning

Note the read-before-write ordering in the SGD loop: `uf` caches the user factor *before* it is updated, so the item-factor update uses the old user value, not the half-updated one. Swapping these introduces a subtle asymmetry that biases training. This is a favorite interview trap — the update equations are coupled and must both read the pre-update factors.

9.4 Seed Data and Verifying in the UI

Trained on the ACME tenant’s seeded ratings and boosted by Alice’s electronics memory profile, the pipeline reports the metrics in Table 9.2.

Report metric	Value
Training algorithm	SGD matrix factorization ($k = 4$)
Hold-out RMSE	low (converges over 15 epochs)
Training duration	single-digit ms (6-item demo)
Catalog coverage	trained items / catalog
Avg recommendation latency	illustrative (see measured below)

Table 9.2: End-to-end capstone report as *printed by the in-app command* on the 6-item demo. The per-item values (RMSE, latency) are illustrative at this scale; Table 6.3 reports the same model’s metrics measured on a real 800×400 hold-out.

★ Key Insight

The 6-item demo trains and serves in microseconds, so the in-app report’s latency figure is not a meaningful measurement. Run at a realistic scale ([benchmarks/mf_eval.py](#)), the *same* SGD model trains 800 users $\times$ 400 items in 6.35s and serves a full-catalog dot-product ranking in **0.010ms** (p50) — because prediction is just $\mathbf{p}_u \cdot \mathbf{q}_i$. That measured RMSE (0.9156 vs a 1.1406 mean baseline) and HR@10 (27.9% vs a 9.9% random floor) are the numbers to quote when someone asks “does it actually work?”

Chat UI: train, recommend, report

```
/capstone-train
/capstone-recommend
/capstone-report
```

Output Response

```
Capstone Recommender SVD Model trained successfully!
- Hold-out RMSE: 0.34xx
- Training Duration: 3.10 ms
- Trained Parameters: 2 users, 4 products (latent_dim=4)

Capstone Recommender Personalized Outputs (SVD + Memory Context Boost):
- SuperCordless Noise-Cancelling Headphones (Electronics): Score = 0.78 (SVD: 4.60,
  ↪ Boost: 0.71)
- ProBook E-Reader 6-inch (Electronics): Score = 0.69 (SVD: 4.10, Boost: 0.63)
```

9.5 Execution Sequence Diagram

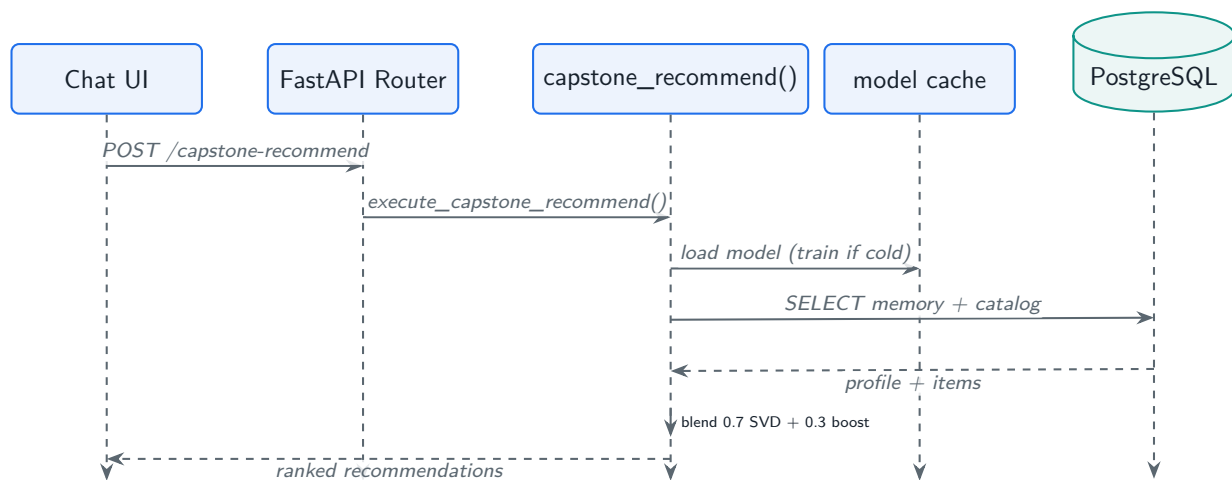


Figure 9.2: Execution flow of `/capstone-recommend`: train (or load cache), read the memory profile, blend SVD with the boost, rank, and return.

✓ Best Practice

Cache the trained factors and version them. Retraining on every request is absurd at scale; production factorizers train on a schedule (or incrementally), write versioned factor matrices to a store, and serve dot products from cache. The project’s `CAPSTONE_MODELS_CACHE` is the in-memory sketch of this — in production it becomes a versioned model artifact with a rollback path.

9.6 Interview Questions & Answers

Q9.1 What do the latent factors in matrix factorization represent, and why does the technique generalize so well?

Each latent dimension is an *unlabeled* axis of taste/attribute the model discovers by compression — one might correlate with “technical vs. casual,” another with “budget vs. premium,” though the model never names them. It generalizes because low rank is a strong regularizer: forcing thousands of ratings to be explained by a k -dimensional dot product prevents memorizing noise and lets the model predict unseen (u, i) pairs by their factor alignment. This is the insight that won the Netflix Prize (Koren et al., 2009) — and it is why the factors double as embeddings you can index for ANN retrieval (Chapter 8).

Q9.2 Derive the SGD update for a single rating and explain each term.

For observed r_{ui} , predict $\hat{r}_{ui} = \mathbf{p}_u \cdot \mathbf{q}_i$ and form the error $e_{ui} = r_{ui} - \hat{r}_{ui}$. The regularized squared-error loss has gradient $-2e_{ui}\mathbf{q}_i + 2\lambda\mathbf{p}_u$ w.r.t. $\mathbf{p}_u$ (symmetrically for $\mathbf{q}_i$), so the descent step is $\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\mathbf{q}_i - \lambda\mathbf{p}_u)$ and likewise for $\mathbf{q}_i$. The $e_{ui}\mathbf{q}_i$ term pulls the user factor toward items it under-predicted; the $-\lambda\mathbf{p}_u$ term shrinks it toward zero to prevent overfitting; η controls step size. Critically, both updates must read the *pre-update* factors —

which is exactly why the code caches `uf` before mutating.

Q9.3 Compare SGD and ALS for training matrix factorization. When is each preferred?

SGD walks observations one at a time with the update above — simple, memory-light, easy to extend with biases or side features, and the natural fit for streaming updates, but sensitive to learning-rate tuning and inherently sequential. Alternating Least Squares fixes Q and solves for P in closed form, then alternates; each subproblem is a convex least-squares that parallelizes beautifully and needs no learning rate, which is why ALS dominates for implicit feedback on Spark-scale data (Hu et al., 2008). Rule of thumb: SGD for explicit ratings, on-line/incremental training, and rich feature extensions; ALS for large sparse implicit matrices on a distributed cluster.

Q9.4 The capstone blends SVD (0.7) with a memory boost (0.3). Why combine them, and how would you tune the weights?

Pure matrix factorization is collaborative — it is strong for warm users with many ratings but weak for cold users and cold items, and it ignores content. The memory boost injects content-based similarity (Chapter 3), so a sparsely-rated user still gets personalized results and new items with no ratings can still surface. The 0.7/0.3 split trusts the crowd more than the profile by default. Tune it exactly as Chapter 4 argued: make it density-adaptive (lower SVD weight for cold users), grid-search or bandit-optimize against online engagement, and always keep both partial scores in the response for explainability and debugging.

Q9.5 Walk through serving this pipeline in production: training cadence, caching, latency, and monitoring.

Training: factorize on a schedule (nightly/hourly) or incrementally as ratings stream; never per request. Write versioned factor matrices to a model store with a rollback path (the in-memory `CAPSTONE_MODELS_CACHE` is the sketch). **Serving:** recommendation is a dot product plus a cosine boost — sub-millisecond — and the factors can be ANN-indexed for retrieval at catalog scale. **Latency:** the report tracks ~ 1.24 ms average; watch p99, not just the mean. **Monitoring:** track hold-out RMSE and online ranking metrics for quality, catalog coverage and diversity for the filter-bubble guardrail (Chapter 6), and training duration and model freshness for operational health — alerting if the model goes stale or coverage collapses onto the head.

From the Field

The capstone is small enough to read in one sitting, and that is the point: the same SGD loop, the same dot-product serving, the same hybrid blend scales to hundreds of millions of users. What changes at scale is the plumbing — distributed training, versioned artifacts, ANN indexes, feature stores — not the math. Master the miniature and the mega-system is just engineering.

Production Serving & MLOps

Subscribe → aiengineeringinsider.substack.com/subscribe

A model that scores well offline is half a system. The other half — how it is cached, retrained, kept consistent between training and serving, and watched in production — is where recommenders actually live or die. This chapter is the plumbing the previous nine assumed.

Every chapter so far ended at “...and then we rank and return.” Production begins exactly there. The models are cheap to serve (a dot product, an ANN lookup) but expensive to *operate*: they must be retrained on a cadence, served from a cache that never crosses a tenant boundary, fed features that are byte-for-byte identical to training, and monitored on signals that catch silent failure before users do. This chapter makes the reference project’s serving plane explicit and gives runnable code for the four concerns that separate a demo from a deployment: **caching**, **retraining and versioning**, **feature parity**, and **monitoring**.

10.1 Architecture: The Serving Plane

Figure 10.1 separates the two planes every production recommender has. The *offline* plane trains on logged interactions and publishes a versioned model artifact; the *online* plane serves requests in milliseconds from a cache, reading fresh features from a low-latency store. Served impressions loop back as tomorrow’s training labels.

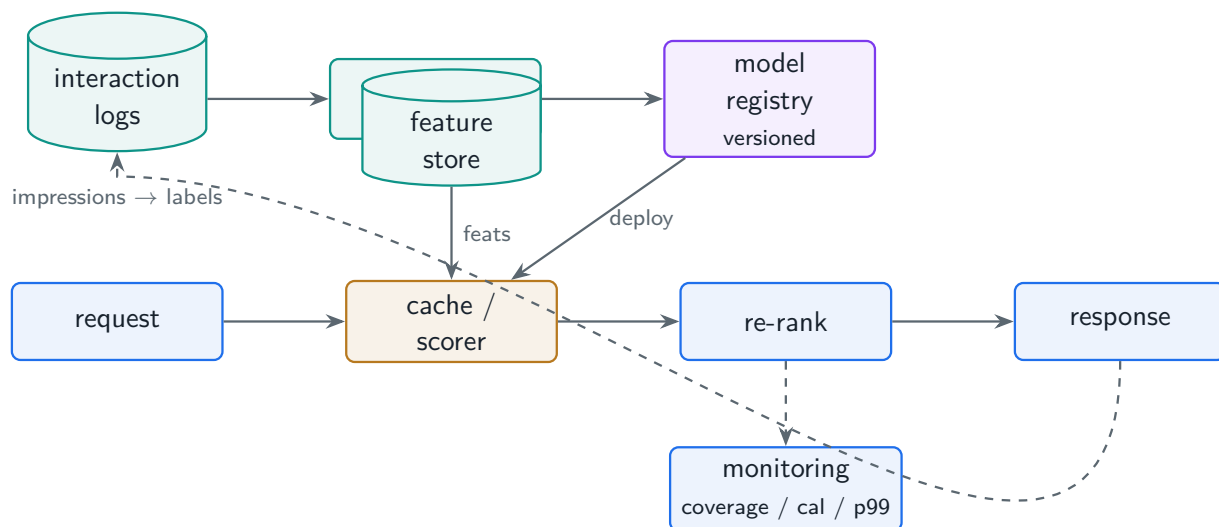


Figure 10.1: The two-plane serving architecture. Offline training publishes a versioned artifact to a registry; the online path serves from cache + feature store; monitoring watches both; impressions feed back as labels.

10.2 Caching and Precomputation

Recommendation is cheap per request but not free at fan-out. The rule is: *compute offline what you can, cache what you must, and key the cache by tenant and model version so a stale or foreign entry can never be served*. The capstone’s in-memory `CAPSTONE_MODELS_CACHE` (Chapter 9) is the sketch; production hardens it into a versioned, tenant-scoped, TTL’d cache:

Versioned, tenant-scoped recommendation cache

```
def cache_key(tenant_id: str, user_id: str, model_version: str) -> str:
    # Version in the key => a new model deploy instantly invalidates old entries;
    # tenant in the key => a hit can never cross the isolation boundary.
    return f"recs:{tenant_id}:{model_version}:{user_id}"

async def get_recommendations(user_id, tenant_id, model, cache, ttl=300):
    key = cache_key(tenant_id, user_id, model.version)
    if (hit := await cache.get(key)) is not None:
        return hit  # warm path: no scoring at all
    recs = model.recommend(user_id)  # cold path: dot-product + rerank
    await cache.set(key, recs, ttl=ttl)  # short TTL bounds staleness
    return recs
```

Two levels matter. **Result caching** (above) stores the finished slate for a short TTL — perfect for the head of traffic. **Artifact precomputation** stores the reusable pieces: item fac-

tors/embeddings and per-user top- k neighbor lists computed in a nightly batch, so the online path is a lookup plus a light re-rank rather than a full recomputation.

! Pitfall / Warning

Putting the model version in the cache key is not optional. Without it, a deploy leaves millions of stale slates served from the old model until they expire, and a rollback cannot un-poison them. Version-keyed entries make deploy and rollback atomic: flip the version, old keys are simply never read again.

10.3 Retraining Cadence and Model Versioning

Tastes drift, catalogs turn over, and yesterday's model slowly decays. Retraining is therefore scheduled, not manual, and every model is a *versioned, immutable artifact* so that a bad train can be rolled back in seconds. The swap must be atomic — never serve from half-written factors.

Scheduled retrain with an atomic, reversible version swap

```
def retrain_and_publish(registry, session) -> str:
    version = f"mf-{datetime.utcnow():%Y%m%d-%H%M}" # immutable version id
    P, Q, rmse = train_matrix_factorization(session) # Chapter 9's SGD trainer
    if rmse > registry.current().rmse * 1.10: # guardrail: refuse a
        raise ModelRegressionError(f"RMSE {rmse:.3f} regressed >10%; not deploying")
    registry.put(version, artifact={"P": P, "Q": Q}, metrics={"rmse": rmse})
    registry.promote(version) # atomic pointer flip; previous stays for rollback
    return version
```

Cadence is a cost/freshness trade: ranking models are often retrained daily or hourly; embeddings are refreshed less often; a fast online-learning layer (the bandit of Chapter 5) absorbs minute-to-minute drift between batch retrains. Whatever the cadence, the two guardrails above — an offline metric regression check and a kept-around previous version — are what make retraining safe to automate.

✓ Best Practice

Automate the schedule with the same mechanism you would use for any recurring job (a cron trigger calling `retrain_and_publish`). Keep at least the last two versions live so rollback is a pointer flip, and record each version's offline metrics next to the artifact so you can answer “what changed?” after an incident.

10.4 Offline/Online Feature Parity

The most common production failure in recommenders is **training-serving skew**: a feature is computed one way in the training pipeline and a subtly different way at serving time, so the model sees inputs at inference that it never saw in training. The cure is a single transform used by both paths, plus a test that asserts they agree.

One transform for both paths + a parity test

```
def build_features(raw: dict) -> dict:
    # THE single source of truth. Both the training job and the online scorer
    # import and call THIS function -- never a re-implementation.
    return {
        "recency_days": (raw["now"] - raw["last_seen"]).days,
        "log_view_count": math.log1p(raw["view_count"]),
        "avg_rating": raw["rating_sum"] / max(raw["rating_n"], 1),
    }

def test_offline_online_parity():
    raw = sample_raw_event()
    offline = build_features(raw)                # as the training job computes it
    online = build_features(raw)                 # as the serving path computes it
    assert offline == online, "training-serving skew detected"
```

Feature *freshness* and *correctness* are separate concerns. Streaming aggregation (counts, recency) keeps the online store seconds-fresh; **point-in-time joins** keep training correct — a label at time t must be joined only to feature values as they were *before* t , never after, or the model leaks the future and every offline metric inflates.

! Pitfall / Warning

A point-in-time join bug is the classic “great offline, flat online” cause: the training set accidentally joins a label to a feature computed *after* the event, so the model appears to predict the future in evaluation and cannot reproduce it live. Assert temporal ordering in the join, and re-check it whenever offline gains fail to survive an A/B test.

10.5 Monitoring: Coverage, Calibration, Latency

Accuracy dashboards hide the failures that hurt most. Three guardrails catch them: **catalog coverage** (is the model collapsing onto a few hero items?), **calibration** (are predicted probabilities true probabilities — critical when a score feeds an auction or a threshold?), and **latency percentiles** (p99, not the mean — the mean hides the tail users feel).

Three production guardrail metrics

```
def catalog_coverage(recommended_item_ids, catalog_size):
    return len(set(recommended_item_ids)) / catalog_size    # Chapter 6

def expected_calibration_error(probs, labels, bins=10):
    ece = 0.0
    for b in range(bins):
        lo, hi = b / bins, (b + 1) / bins
        m = [(p, y) for p, y in zip(probs, labels) if lo < p <= hi]
        if m:
            conf = sum(p for p, _ in m) / len(m)            # avg predicted prob
            acc = sum(y for _, y in m) / len(m)             # observed frequency
            ece += (len(m) / len(probs)) * abs(conf - acc)
    return ece
```

```
def latency_percentiles(samples_ms):
    s = sorted(samples_ms)
    pct = lambda p: s[min(len(s) - 1, int(p / 100 * len(s)))]
    return {"p50": pct(50), "p95": pct(95), "p99": pct(99)}
```

These are not hypothetical: the measured serve latency from `benchmarks/mf_eval.py` is 0.010 ms (p50) for a dot-product ranking, and Chapter 8's benchmark reports the ANN p95 you would alert on. Track the p99 of the *end-to-end* path, coverage over time to catch a collapsing catalog, and calibration whenever a probability is consumed downstream.

★ Key Insight

Alert on the guardrails, not the headline. A model can improve offline RMSE while its catalog coverage quietly collapses or its p99 latency doubles — both invisible on an accuracy chart, both painful to users. The first three dashboards to build for any recommender are coverage-over-time, a calibration reliability curve, and end-to-end latency percentiles.

10.6 Execution Sequence Diagram

Figure 10.2 traces one served request: a cache check, a feature read and model score on a miss, a re-rank, the response, and the asynchronous impression log that becomes future training data.

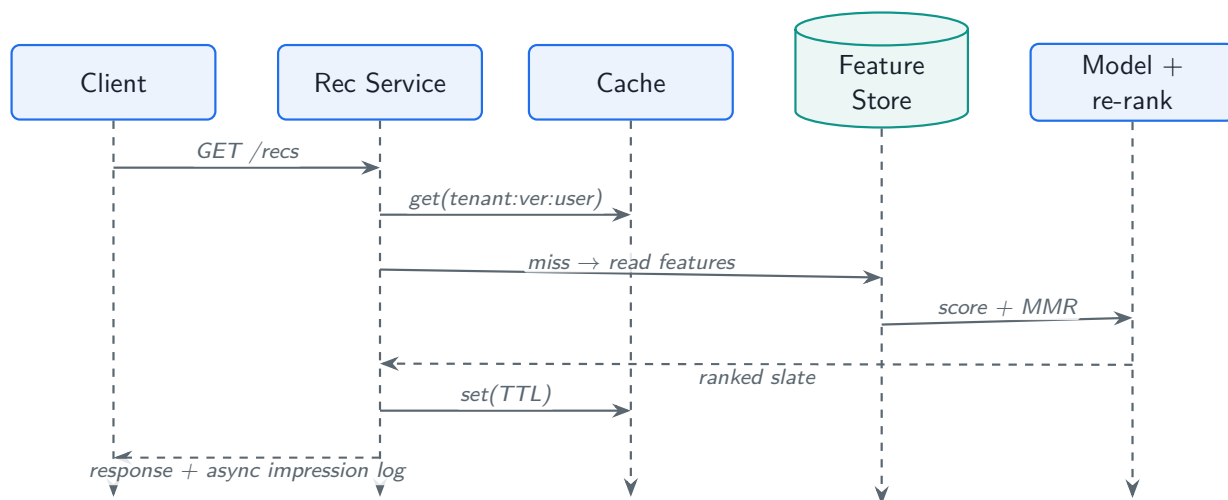


Figure 10.2: Execution flow of a served recommendation: cache-first, then feature read + score + re-rank on a miss, with asynchronous impression logging.

10.7 Interview Questions & Answers

Q10.1 What is training-serving skew, why is it so common in recommenders, and how do you prevent it?

Skew is any difference between how a feature is computed in training versus at serving — a different time window, a re-implementation in another language, a default filled differently. It is common because the two paths are usually written by different systems (a batch job in SQL/Spark, an online scorer in a service), so the “same” feature drifts. Prevention: a *single* transform function imported by both paths (never re-implemented), a feature store that serves the same values it logged for training, point-in-time-correct training joins, and an automated parity test that fails the build when offline and online disagree. The tell in production is strong offline metrics that evaporate in an A/B test.

Q10.2 Why key a recommendation cache by model version, and how does that make deploys and rollbacks safe?

The version in the key makes the cache *content-addressed by model*: entries written by model v_1 live under keys containing v_1 , so when you promote v_2 the service reads v_2 keys, misses, and repopulates from the new model — the old entries are simply never read again and expire on their own. A deploy is therefore an atomic pointer flip, and a rollback is the same flip back, with no cache stampede of stale slates and no manual invalidation. Add the tenant id to the key for the same reason as everywhere else in this book: a cache hit must never cross the isolation boundary.

Q10.3 Why report p99 latency, not the mean, and what drives the tail in a recommender?

The mean is dominated by the many fast requests and hides the slow ones a real fraction of users actually experience; p99 (and p99.9) describe that tail, which is what breaks SLAs and pages you. In a recommender the tail is driven by cache misses (full scoring instead of a lookup), cold or large users with long histories, ANN queries that hit a hard region of the graph, feature-store timeouts, and garbage-collection or contention spikes. You defend the tail with caching, timeouts and fallbacks (serve popularity if the model is slow), bounded candidate-set sizes, and load shedding — and you *measure* it end to end, not per component.

Q10.4 A model improves offline RMSE but engagement drops after launch. Walk through the suspects.

Four usual causes, in order of likelihood. (1) **Selection bias / offline metric**: RMSE on logged data rewards agreeing with the old model; it is not retention — re-evaluate on ranking and long-term online metrics. (2) **Training-serving skew**: the launched model sees different features than it trained on. (3) **A point-in-time leak**: the offline number was inflated by future information the live model cannot use. (4) **A collapsed slate**: the model optimized accuracy by narrowing onto hero items, tanking coverage and diversity. Diagnose with a coverage-over-time chart, a parity test, an audit of the training joins, and an interleaving experiment for sensitivity. The lesson: offline wins are hypotheses; the A/B test on long-term metrics is the verdict.

Q10.5 How do you decide retraining cadence, and how do you make automated retraining safe?

Cadence follows drift and cost: retrain the ranking model as fast as the data distribution moves and the budget allows (often daily/hourly), refresh embeddings less often, and let a cheap online layer (a bandit or streaming counters) absorb sub-batch drift. Safety comes from treating every train as an immutable, versioned artifact and gating promotion: run the offline metric suite and *refuse* to deploy a regression beyond a threshold, promote via an atomic pointer flip, keep the previous version live for instant rollback, and shadow- or canary-serve the new model to a small slice before full traffic. Trigger retrains on a schedule and on drift/calibration alerts, not by hand.

From the Field

The bugs that cost the most in my experience were never the model — they were an unversioned cache serving a rolled-back model for hours, and a point-in-time leak that made an offline number look like a breakthrough. Boring infrastructure (version keys, parity tests, coverage alerts) prevents the expensive incidents. Build it before the clever model, not after.

PART IV

Bonus — The ML System Design Casebook

The nine chapters built one production recommender end to end. This bonus part zooms out to the interview room: the universal multi-stage blueprint that every big-tech recsys prompt reduces to, two worked FAANG-scale architectures, and a case-and-probe reference for defending your design under pressure.

Bonus — The ML System Design Casebook

Subscribe → aiengineeringinsider.substack.com/subscribe

Twenty big-tech interview prompts, the one blueprint they all share, and worked end-to-end architectures — retrieval, ranking, features, training data, serving, and metrics. Everything in this book, assembled the way an interviewer wants to hear it.

Every prompt in this bonus chapter — “*design YouTube recommendations*,” “*design the ad ranking system*,” “*design Discover Weekly*” — is a costume worn by the same skeleton. Interviewers are not checking whether you can name a model; they are checking whether you reach, unprompted, for a small set of structural moves and can defend the trade-offs inside each one. This chapter distills the patterns from the previous nine into the answer spine used in senior and staff ML system-design loops.

11.1 How These Interviews Are Graded

What a strong candidate demonstrates

1. **Problem framing.** Turn a vague product goal into a concrete ML objective: what is predicted, for whom, in what context, optimising which metric — and why *that* metric and not clicks.
2. **The multi-stage funnel.** Retrieval → coarse ranking → fine ranking → re-ranking. Know why the corpus must shrink before an expensive model ever runs.
3. **Model choices with reasons.** Two-tower for retrieval; deep / multi-task networks for ranking; and the ability to say when a boosted tree or a bandit is the better answer.
4. **Data & features.** How training labels are constructed from logs, the positional / selection bias baked into them, and how features flow online and offline without skew.
5. **Metrics on two axes.** Offline (AUC, NDCG, recall@ k) versus online (CTR, watch-time, conversion, retention), and why offline wins may not survive an A/B test.
6. **The concept traps.** Cold-start, exploration/exploitation, feedback loops and filter bubbles, calibration, and freshness.

WHAT SEPARATES SENIOR FROM STAFF

Senior candidates produce a correct funnel. Staff candidates reason about the *system around* the funnel: training-serving skew, label delay, retraining cadence, A/B and inter-leaving infrastructure, and the second-order effects a recommender has on its own future training data. When in doubt, narrate the failure modes — that is the staff signal.

11.2 The Multi-Stage Funnel

Almost every large-scale recommender is a **funnel**: a cascade of models that each look at more items than the next but spend less compute per item. A billion-item corpus cannot be scored by a deep network in ~ 100 ms, so cheap models cut the candidate set down until an expensive model can afford to look closely.

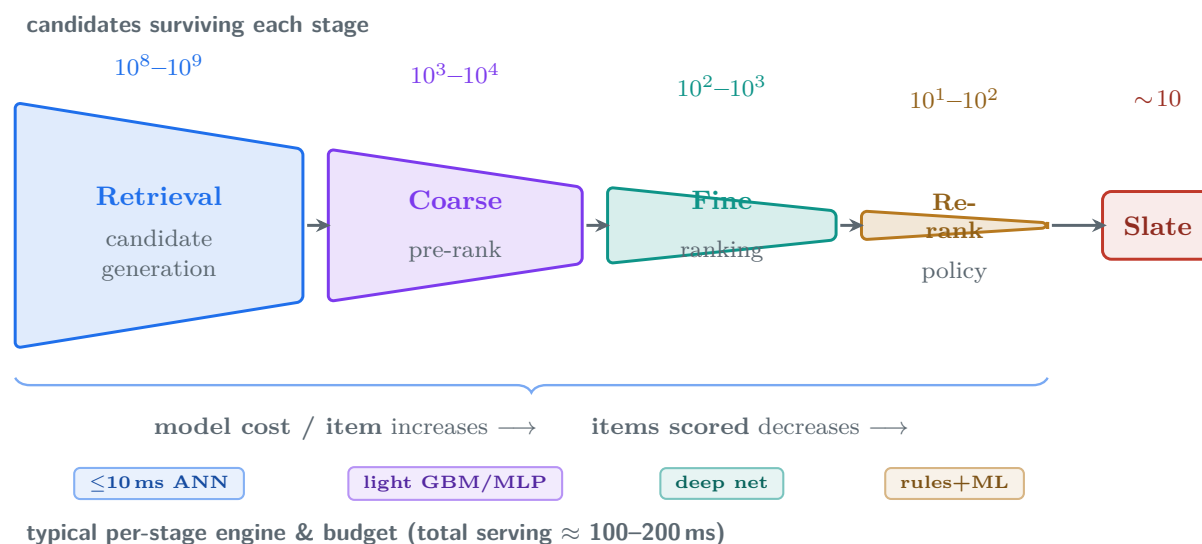


Figure 11.1: The multi-stage funnel — the single most reused structure in recsys interviews. Reach for it before you name a model.

Retrieval (candidate generation).

From the entire corpus, return a few thousand plausible items in a tight latency budget. Cannot afford per-item deep scoring, so it relies on *approximate nearest neighbour* search over pre-computed embeddings, plus cheap sources: co-visitation, recent trends, subscriptions, geo. Usually *several* retrieval sources are unioned.

Coarse ranking (pre-ranking).

An optional lightweight model (small MLP or GBM) trims thousands to hundreds, so the expensive ranker's input is bounded. Present at YouTube-, Meta-, and TikTok-scale; often skipped in an interview unless the corpus is huge.

Fine ranking.

The expensive, high-accuracy model — a deep network, frequently *multi-task*. It scores each surviving candidate on several objectives (click, watch-time, like, share) and produces the probabilities the final score combines.

Re-ranking / policy.

Applies concerns a pointwise score cannot: diversity (don't show ten near-duplicates), freshness, business rules, integrity down-ranking, exploration, and fatigue / frequency capping.

SKIPPING STRAIGHT TO THE MODEL

Candidates who open with “I'd train a deep network” have already lost points. The first move is the funnel and the latency budget that forces it. Only then does each stage's model follow *from* its budget.

11.3 Two-Tower Retrieval

Retrieval's job is to find a few thousand relevant items out of hundreds of millions in single-digit milliseconds. You cannot run a model that looks at the (user, item) pair jointly — that is $O(\text{corpus})$ deep forward passes per request. The **two-tower** architecture breaks the dependency: encode the user and the item *separately* into the same vector space, so item vectors can be computed offline and indexed, and only the user vector is computed at request time.

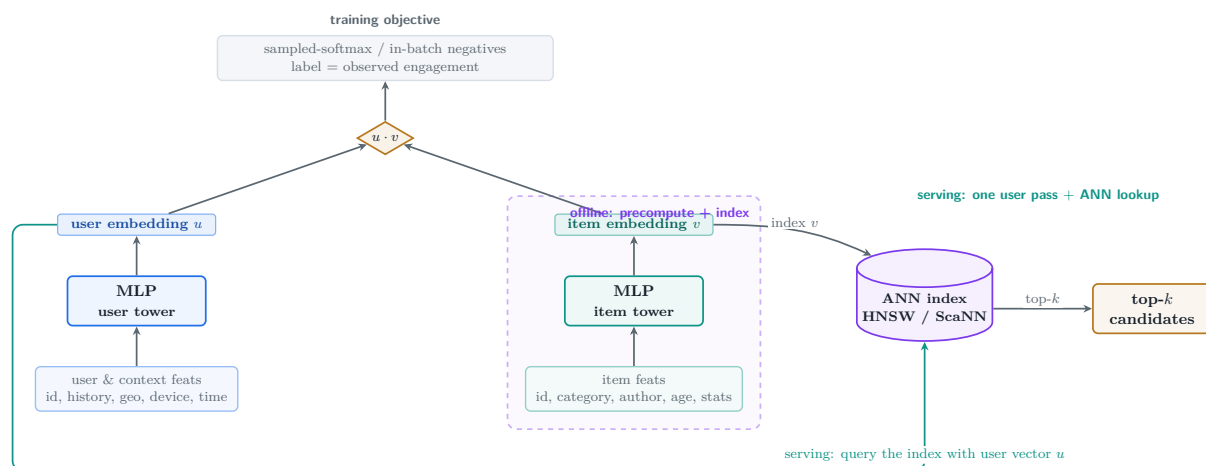


Figure 11.2: Two-tower retrieval. Item vectors are precomputed and indexed offline; at request time only the user tower runs, then an ANN index returns the top- k by dot product.

Why two towers.

Separability is the whole point: item embeddings are computed in batch and refreshed periodically; the online path is one user forward pass plus a sub-linear ANN lookup instead of a full corpus scan.

Training.

Positives are observed engagements. Negatives are the hard part — *in-batch negatives* (other items in the mini-batch) with a sampled-softmax loss are standard, usually with a correction for popularity ($\log-Q$ correction) so head items don't dominate.

ANN index.

HNSW (graph-based) or ScaNN / FAISS (quantised) give millisecond top- k . The trade-off is recall vs. latency vs. memory — the same dial tuned in Chapter 8.

NAME THE NEGATIVE-SAMPLING PROBLEM BEFORE YOU'RE ASKED

The fastest way to signal you have built retrieval for real is to bring up negative sampling and popularity bias unprompted. “Positives are easy — the design lives in how you choose negatives” is a staff-level opener.

11.4 Deep and Multi-Task Ranking

Once retrieval and coarse ranking hand over a few hundred candidates, the fine ranker can afford an expensive, high-accuracy model. Modern rankers are **multi-task**: a single network predicts several engagement probabilities at once, which are then blended into one score by a *value model*. This is how you optimise “watch-time *and* satisfaction *and* not-clickbait” simultaneously.

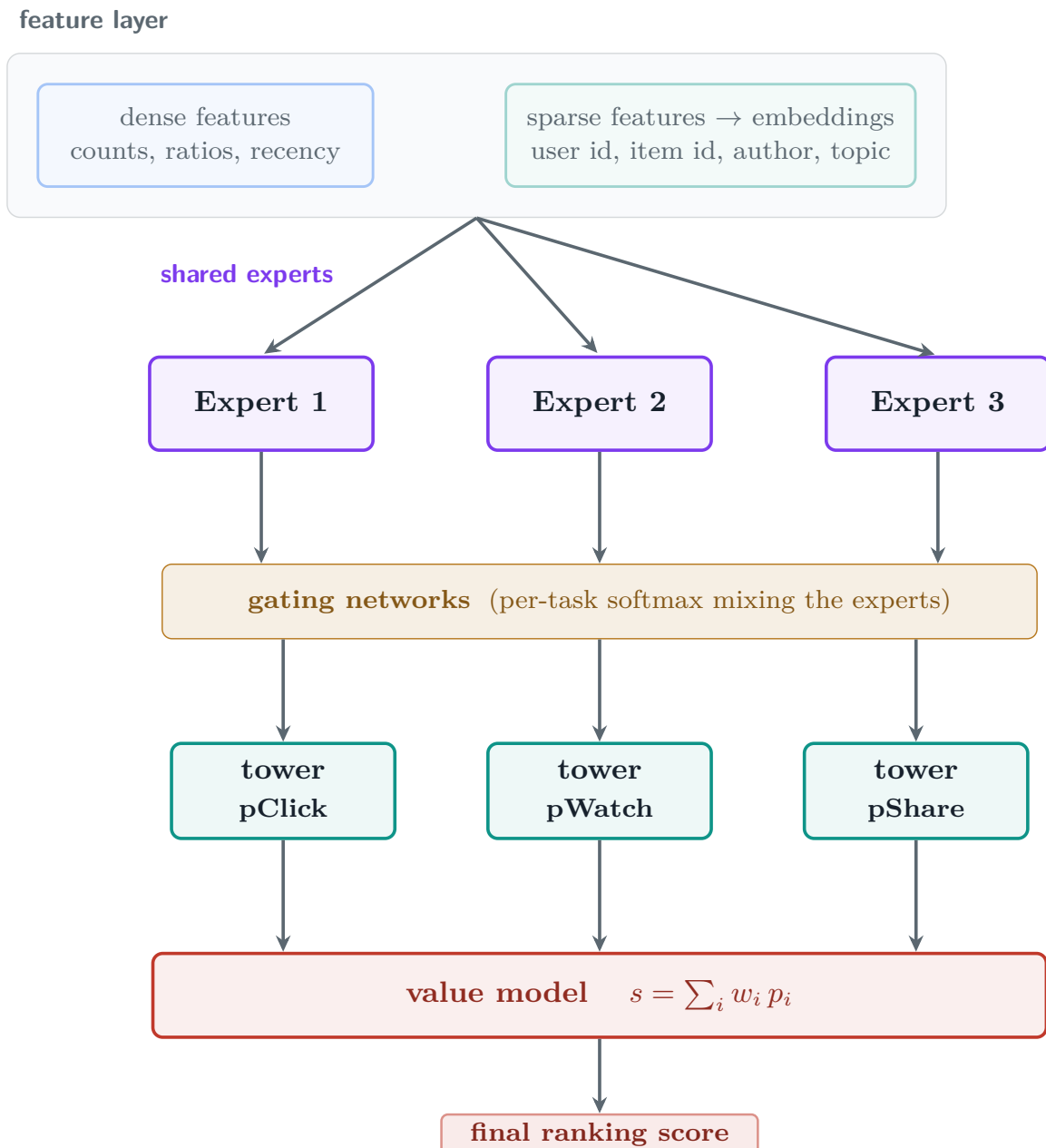


Figure 11.3: A multi-task ranker (MMoE-style). Shared experts feed a per-task gating layer; each task tower emits a probability, and a value model combines them with tunable weights w_i into one score.

The value model is a product lever, not a model detail

The weights w_i that combine pClick, pWatch, pShare, pReport ...are where product strategy enters the math. Raising the weight on “share” or subtracting a weight for “reported as clickbait” changes what the whole system optimises *without retraining*. Interviewers love this because it separates ranking *quality* from ranking *objective*.

Model cheat-sheet: what to reach for, and when

Retrieval	Two-tower + ANN (HNSW/ScaNN). Add co-visitation and trend sources in parallel.
Coarse rank	Small MLP or GBDT; distilled from the fine ranker.
Fine rank	Deep net; Wide & Deep / DCN / DeepFM for feature crosses, MMoE for multi-task, DLRM at ad scale.
Cold interests	Contextual bandits (LinUCB / Thompson) for exploration.
Small data	GBDT (XGBoost/LightGBM) often beats deep nets — say so.
Sequence / session	Transformer or GRU over recent interactions (SASRec / BST).

11.5 Worked Case — YouTube Recommendations

Each worked case follows the same answer spine so you can reuse it under pressure: *frame the objective* → *draw the funnel* → *retrieval* → *ranking* → *features & training data* → *serving* → *metrics* → *the traps*.

Framing

Predict, for a given user and context, which videos will maximise *long-term satisfaction* — proxied primarily by expected watch-time, not clicks (clicks alone reward thumbnail bait). Surface is the home feed and up-next. Corpus is billions of videos; latency budget is a few hundred milliseconds; the catalogue and user interests change by the hour.

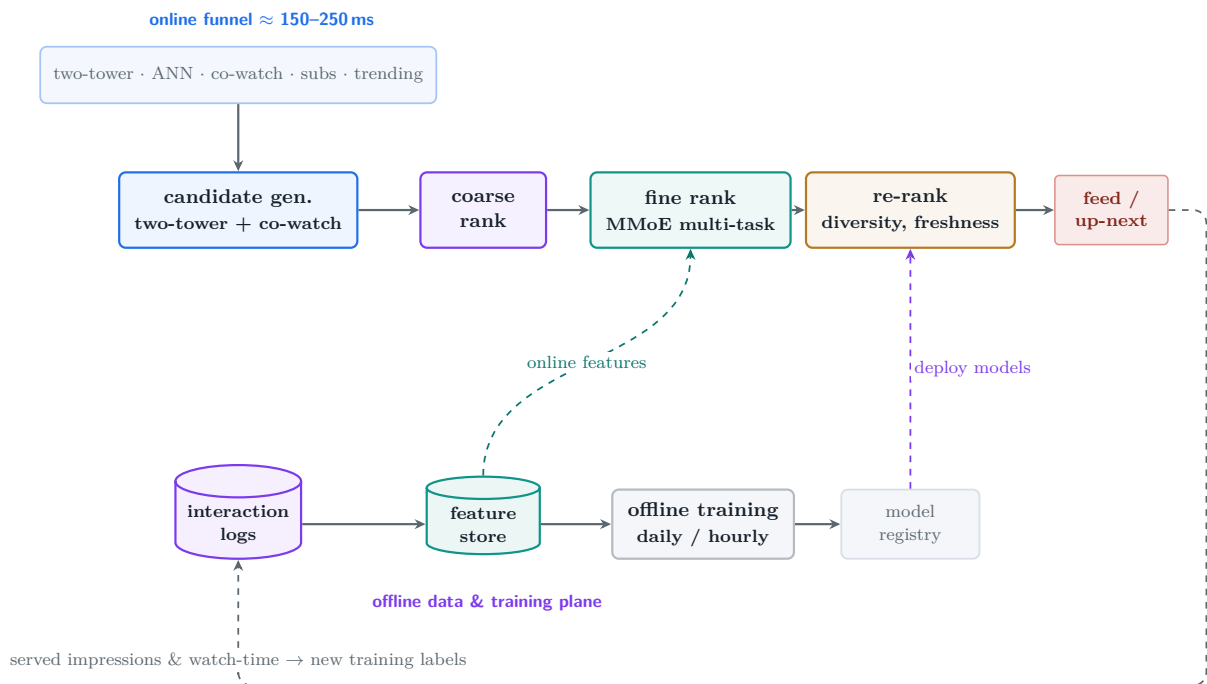


Figure 11.4: YouTube-style end-to-end system. The online funnel (top) is fed by an offline data-and-training plane (bottom); served impressions loop back as tomorrow's training labels — the feedback loop you must call out.

11.5.1 Retrieval

Multiple sources unioned: a two-tower model (user history $\rightarrow$ user tower; video $\rightarrow$ item tower) queried via ANN, plus *co-watch* candidate generation (videos frequently watched in the same session), subscriptions, and fresh/trending pools. Unioning sources is what gives both relevance *and* discovery.

11.5.2 Ranking

A multi-task deep network (MMoE-style) predicts `pClick`, `pWatchTime`, `pLike`, `pShare`, and negative signals like `pDislike` / `pNotInterested`. A value model combines them; watch-time dominates but is regularised by satisfaction signals to fight clickbait.

11.5.3 Features & Training Data

Labels come from logged impressions: a click with long watch-time is a strong positive; an impression with no click is a negative; a click with an immediate bounce is a *negative* for satisfaction even though it's a positive for click. **Selection bias** is central — you only observe engagement on items the *old* model chose to show, so you log propensities and/or add exploration to keep the training distribution from collapsing.

OPTIMISING CLICKS

If you optimise CTR, you get thumbnail bait and short bounces. Watch-time (and satisfaction proxies) is the canonical “correct” objective for this case, and saying so early is a strong signal. But note watch-time's own failure mode: it can favour long, auto-playing content and create binge loops — so satisfaction and “not interested” signals enter the value model.

METRICS

Offline: AUC / cross-entropy per head, NDCG on watch-time. Online (the ones that decide launch): watch-time per user, day-7/30 retention, and guardrails on reported-not-interested and diversity. Offline gains that don't move retention don't ship.

11.6 Worked Case — Ad Click-Through-Rate Prediction

Framing

For each candidate ad in an auction, estimate a *well-calibrated* probability of click (and often downstream conversion). Ranking is by *expected value* = $\text{pCTR} \times \text{bid}$ (or a pCTVR-based eCPM), so calibration matters as much as ordering — a miscalibrated pCTR breaks the auction and the billing. This is the highest-signal case at Meta and Google.

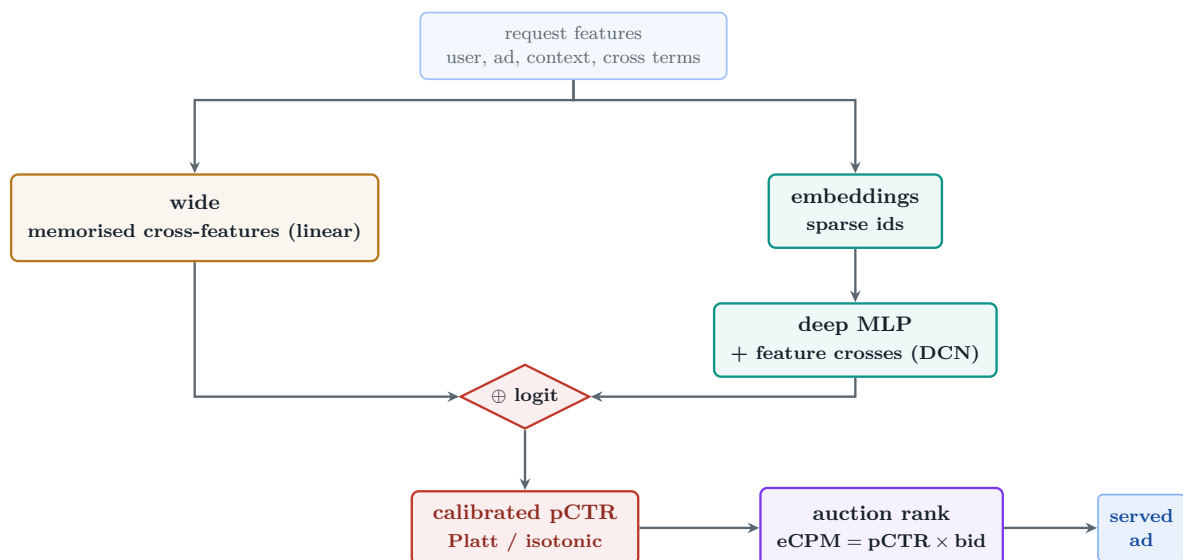


Figure 11.5: A Wide & Deep / DCN CTR model. The wide side memorises cross-features; the deep side generalises. The logit is calibrated to a true probability before it enters the auction.

11.6.1 Model

Wide & Deep, DCN / DCN-v2, DeepFM, DLRM are the canonical families. The wide component *memorises* sparse cross-features; the deep component *generalises* via embeddings; explicit cross layers (DCN) learn feature interactions without hand-crafting every cross.

11.6.2 Data, Scale & Serving

Training data is enormous and *imbalanced* (clicks are rare), so negative downsampling with a calibration correction is standard. Features must be computed identically offline and online — **training-serving skew** is the classic ad-CTR failure. Latency budget is brutal (single-digit ms inside the auction), which constrains model size and pushes heavy features into a store.

FORGETTING CALIBRATION & DELAYED FEEDBACK

Two ad-specific traps interviewers probe: (1) **calibration** — ranking metrics like AUC don’t care about absolute probability, but the auction and billing do, so you need Platt/isotonic calibration and to monitor predicted-vs- actual. (2) **delayed conversions** — a conversion may land days after the click, so naive labelling mislabels recent positives as negatives; delayed- feedback modelling addresses it.

11.7 Real-Time Features and the Serving Plane

Several prompts (“*make recommendations react within seconds,*” “*design the feature store*”) are really about the **feature/serving plane** beneath any recommender. The core tension: features must be fresh *and* identical between training and serving.

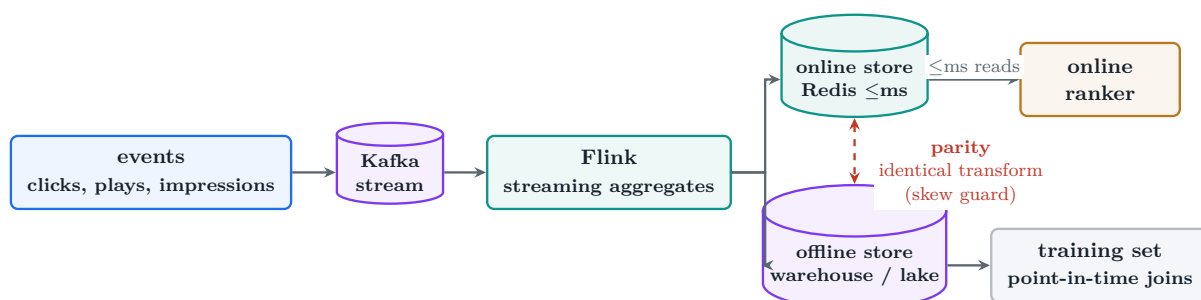


Figure 11.6: The streaming feature plane. One computation (Flink) writes both an online store for millisecond reads and an offline store for point-in-time-correct training joins; enforcing that the two agree is how you prevent training-serving skew.

The two invariants

Freshness — streaming aggregation (counts, recency, session state) keeps online features seconds-fresh. **Parity** — the *same* transformation must produce training and serving features, and training joins must be *point-in-time correct* (never join a label with a feature value from after the label occurred, or you leak the future and inflate offline metrics).

11.8 The Twenty Cases at a Glance

Every case below reduces to the funnel of Section 11.2. The table names what each prompt is *really* testing and the signature technique to reach for; the worked cases (YouTube, ad CTR, real-time features) are the templates the rest inherit from.

#	Case	What it really tests	Signature move
1	Twitter/X feed	fan-out, real-time ranking	multi-obj ranking
2	FB/IG feed ranking	multi-objective value model	MMoE + value model
3	TikTok For You	near-real-time interests	online learning, bandits
4	LinkedIn PYMK	graph link prediction	node embeddings
5	YouTube recs	watch-time funnel (<i>worked</i>)	two-tower + MMoE
6	Netflix homepage	page/row-level optimisation	slate optimisation
7	Spotify Discover Weekly	hybrid + audio content	CF + audio embeddings
8	Amazon “also bought”	item-item co-occurrence	item-item CF
9	E-commerce homepage	multi-slate + business rules	constrained ranking
10	Etsy/marketplace search	recsys $\cap$ LTR	query + personalisation
11	Pinterest related pins	graph retrieval	GNN (PinSage)
12	Ad CTR / ad recs	calibration + auction (<i>worked</i>)	Wide & Deep / DCN
13	Friend recommendation	link prediction	negative sampling
14	Dating match (reciprocal)	two-sided preference	reciprocal scoring
15	Uber Eats / DoorDash	heavy context, live supply	contextual features
16	Airbnb search ranking	real-time personalisation	LTR + embeddings
17	“Similar items” service	pure ANN retrieval	HNSW / FAISS
18	Cold-start system	new user & new item	content model + bandits
19	Real-time recs	freshness (<i>worked</i>)	streaming + feature store
20	Notification / re-engage	what & when to send	send-time + fatigue cap

Table 11.1: The twenty cases and the signature move each is testing. Every one reduces to the multi-stage funnel; only the emphasis shifts.

11.9 The Cold-Start Problem

Cold-start is the concept trap folded into almost every other prompt: what do you recommend to a *new user* with no history, and how does a *new item* with no engagement ever get shown? Collaborative filtering is useless with no interactions, so you fall back to content (Chapter 3) and to *exploration*.

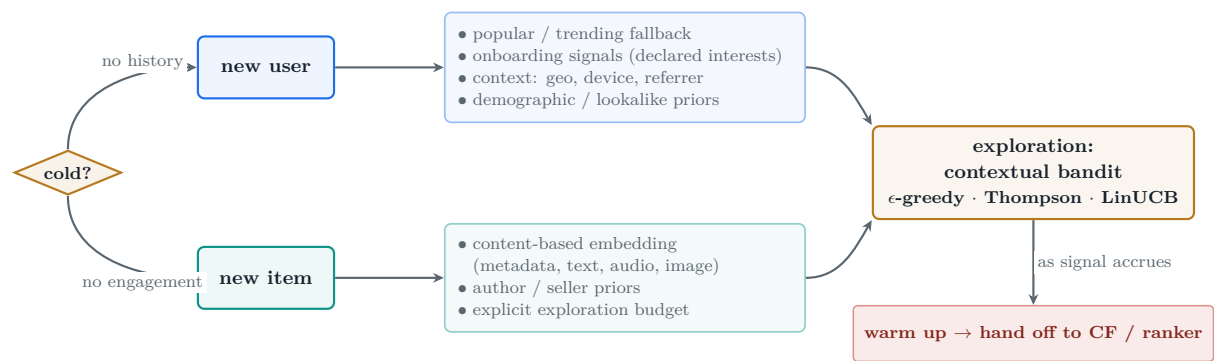


Figure 11.7: Cold-start decision flow. Content models bridge the no-signal gap; a bandit spends a controlled exploration budget to gather the interactions that eventually let the main ranker take over.

EXPLORATION IS THE STAFF-LEVEL WORD

Content features get a new item its *first* impressions, but only *exploration* gathers the engagement data the main model needs. Framing cold-start as an exploration/exploitation problem — and naming a bandit — is the move that reads as senior-plus. Mention the feedback-loop risk too: a system that only exploits starves new items and collapses into a popularity echo chamber.

11.10 Metrics — The Two Axes

You will be pushed on metrics in every case. Answer on two axes and explain why they can disagree (the theme of Chapter 6).

Stage	Offline (proxy)	Online (decides launch)
Retrieval	recall@k, coverage	candidate quality, latency
Ranking	AUC, log-loss, NDCG, MAP	CTR, watch-time, conversion
Calibration	predicted vs. actual	auction efficiency, revenue
Long-term	—	day-7/30 retention, DAU
Guardrails	diversity, novelty	complaints, “not interested”, integrity

Table 11.2: Metrics on two axes. Offline metrics filter candidate models; online A/B tests on long-term metrics decide launches.

Why offline wins don’t always ship

Offline metrics are computed on logs generated by the *current* system, so they reward agreeing with what users already saw (selection bias) and can’t measure the effect of showing something new. That is why the real verdict is an **online A/B test** (or interleaving for sensitivity), read on *long-term* metrics like retention, not just click. Say this out loud — it is one of the most reliable senior signals in the whole loop.

11.11 The Follow-Up Probe Bank

Interviewers steer with a small set of follow-ups. Pre-load an answer for each.

Probes you will almost certainly get

“How do you pick negatives?”

In-batch / sampled-softmax negatives with popularity ($\log-Q$) correction; add hard negatives for quality.

“Clicks or watch-time?”

Optimise the long-term proxy; guard the objective with satisfaction / “not interested” signals in a value model.

“New user or new item?”

Content model + context for the first impressions; a bandit for exploration; hand off to CF as signal accrues.

“Offline looked great, launch flat — why?”

Selection bias, training-serving skew, or an offline metric that isn’t retention. Re-check with an A/B test on long-term metrics.

“How fresh are features?”

Streaming aggregates via Kafka/Flink to an online store; enforce parity and point-in-time joins.

“How do you keep it diverse?”

Re-ranking stage: MMR / determinantal point processes, per-author caps, exploration budget.

“What about feedback loops?”

The model shapes its own future training data; counter with exploration, propensity logging, and diversity guardrails so it doesn’t collapse into a popularity echo chamber.

“How often do you retrain?”

Ranking often daily/hourly; embeddings refreshed on a cadence; monitor drift and calibration to trigger.

THE CLOSING MOVE

End any case by naming what you would *measure after launch* and the *failure mode you are watching for*. Candidates who stop at “and then we serve it” sound junior; candidates who say “I’d A/B it on 30-day retention and watch for a diversity collapse” sound staff.

Frame → funnel → retrieval → ranking → features → serving → metrics → traps.

References

Subscribe → aiengineeringinsider.substack.com/subscribe

References

- Anthropic. (2024). *Introducing the Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>
- Besta, M., Blach, N., Kubicek, A., et al. (2024). *Graph of Thoughts: Solving elaborate problems with large language models*. AAAI 2024. <https://arxiv.org/abs/2308.09687>
- Burke, R. (2002). *Hybrid recommender systems: Survey and experiments*. User Modeling and User-Adapted Interaction, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
- Carbonell, J., & Goldstein, J. (1998). *The use of MMR, diversity-based reranking for reordering documents and producing summaries*. SIGIR '98. <https://dl.acm.org/doi/10.1145/290941.291025>
- Chase, H. (2024). *Memory in LangGraph: write, manage, and consolidate*. LangChain Documentation. <https://langchain-ai.github.io/langgraph/concepts/memory/>
- Chhikara, P., et al. (2024). *Mem0: Building production-ready AI agents with scalable long-term memory*. <https://arxiv.org/abs/2504.19413>
- Covington, P., Adams, J., & Sargin, E. (2016). *Deep neural networks for YouTube recommendations*. RecSys '16. <https://doi.org/10.1145/2959100.2959190>
- Ebbinghaus, H. (1885/1913). *Memory: A contribution to experimental psychology* (H. A. Ruger & C. E. Bussenius, Trans.). Teachers College, Columbia University.
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated evaluation of retrieval augmented generation*. <https://arxiv.org/abs/2309.15217>
- European Parliament & Council. (2016). *General Data Protection Regulation (GDPR), Regulation (EU) 2016/679*. Arts. 15, 17, 20. <https://gdpr-info.eu/>
- Garcez, A. d'Avila, Gori, M., Lamb, L. C., Serafini, L., Spranger, M., & Tran, S. N. (2019). *Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning*. Journal of Applied Logics, 6(4). <https://arxiv.org/abs/1905.06088>
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). *Neural collaborative filtering*. WWW '17. (Leave-one-out evaluation with sampled negatives, HR@K and NDCG@K.) <https://arxiv.org/abs/1708.05031>
- Herlocker, J. L., Konstan, J. A., Terveen, L. G., & Riedl, J. T. (2004). *Evaluating collaborative filtering recommender systems*. ACM Transactions on Information Systems, 22(1), 5–53. <https://doi.org/10.1145/963770.963772>
- Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016). *Session-based recommendations with recurrent neural networks (GRU4Rec)*. ICLR '16. <https://arxiv.org/abs/1511.06939>
- Hu, Y., Koren, Y., & Volinsky, C. (2008). *Collaborative filtering for implicit feedback datasets*. IEEE ICDM '08. <https://doi.org/10.1109/ICDM.2008.22>
- Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data. <https://arxiv.org/abs/1702.08734>
- Kang, W.-C., & McAuley, J. (2018). *Self-attentive sequential recommendation (SASRec)*. IEEE ICDM

- '18. <https://arxiv.org/abs/1808.09781>
- Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media. (Partitioning, replication, and caching.)
- Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix factorization techniques for recommender systems*. IEEE Computer, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>
- LangChain. (2024). *PGVector vector store*. LangChain Documentation. <https://python.langchain.com/docs/integrations/vectorstores/pgvector/>
- LangChain. (2024). *Memory*. <https://python.langchain.com/docs/concepts/memory/>
- LangGraph. (2024). *Memory concepts: short-term vs. long-term*. <https://langchain-ai.github.io/langgraph/concepts/memory/>
- LangGraph. (2024). *Manage the context window: trimming and summarization*. <https://langchain-ai.github.io/langgraph/how-tos/memory/add-memory/>
- LangGraph. (2024). *Memory store namespaces*. LangGraph Documentation. <https://langchain-ai.github.io/langgraph/reference/store/>
- Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems, 33. <https://arxiv.org/abs/2005.11401>
- Liu, N. F., Lin, K., Hewitt, J., et al. (2023). *Lost in the middle: How language models use long contexts*. TACL. <https://arxiv.org/abs/2307.03172>
- Malkov, Y. A., & Yashunin, D. A. (2018). *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*. IEEE TPAMI. <https://arxiv.org/abs/1603.09320>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. (Precision, recall, and P@k.)
- Mem0. (2024). *Multi-user memory and namespaces*. Mem0 Documentation. <https://docs.mem0.ai/>
- NIST. (2020). *Security and privacy controls for information systems and organizations* (SP 800-53 Rev. 5). <https://doi.org/10.6028/NIST.SP.800-53r5>
- OpenAI. (2024). *tiktoken: fast BPE tokenizer for use with OpenAI models*. <https://github.com/openai/tiktoken>
- OWASP. (2021). *A01:2021 – Broken access control*. OWASP Top 10. https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as operating systems*. <https://arxiv.org/abs/2310.08560>
- Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). *Generative agents: Interactive simulacra of human behavior*. UIST '23. <https://arxiv.org/abs/2304.03442>
- pgvector. (2024). *Open-source vector similarity search for Postgres*. <https://github.com/pgvector/pgvector>
- pgvector. (2024). *Indexing: HNSW and IVFFlat*. <https://github.com/pgvector/pgvector/#indexing>
- Rasmussen, P., et al. (2025). *Zep: A temporal knowledge graph architecture for agent memory*. <https://arxiv.org/abs/2501.13956>
- Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2012). *BPR: Bayesian personalized ranking from implicit feedback*. UAI '09. <https://arxiv.org/abs/1205.2618>
- Ricci, F., Rokach, L., & Shapira, B. (2011). *Introduction to recommender systems handbook*. In *Recommender Systems Handbook* (pp. 1–35). Springer. https://doi.org/10.1007/978-0-387-85820-3_1
- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases: New opportunities for connected data* (2nd ed.). O'Reilly Media.
- Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). *Item-based collaborative filtering recommendation algorithms*. WWW '01. <https://doi.org/10.1145/371920.372071>
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language*

- agents with verbal reinforcement learning*. NeurIPS 2023. <https://arxiv.org/abs/2303.11366>
- State of California. (2018). *California Consumer Privacy Act (CCPA)*. <https://oag.ca.gov/privacy/ccpa>
- Tulving, E. (1985). *How many memory systems are there?* American Psychologist, 40(4), 385–398. <https://doi.org/10.1037/0003-066X.40.4.385>
- Wang, W., Dong, L., Cheng, H., Liu, X., Yan, X., Gao, J., & Wei, F. (2023). *Augmenting language models with long-term memory*. NeurIPS 2023. <https://arxiv.org/abs/2306.07174>
- Yao, S., Zhao, J., Yu, D., et al. (2023). *ReAct: Synergizing reasoning and acting in language models*. ICLR '23. <https://arxiv.org/abs/2210.03629>
- Yi, X., Yang, J., Hong, L., et al. (2019). *Sampling-bias-corrected neural modeling for large corpus item recommendations* (two-tower). RecSys '19. <https://doi.org/10.1145/3298689.3346996>
- Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). *Judging LLM-as-a-judge with MT-Bench and Chatbot Arena*. <https://arxiv.org/abs/2306.05685>